



[Trump Tweets, Sentiment, and Crude Oil Price Movement: An Applied NLP and Machine Learning Project]

Authors

[Zaid Al Raggad], [202120482]
[Mahmood Sanwar], [202210699]
[Ahmad Al Junaidi],[202120421]

Supervised by
Dr. Ayman Mansour

Course
Academic Year
2025/2026

Date
2026



Table of Contents

1. Abstract
2. Acknowledgment
3. Business Intelligence Project Description and Objectives
4. Data Research and Acquiring Effort
5. Links to Raw Data
6. Data Description and Understandings
7. Data Primary Cleaning and Transformation
8. Data Visualization and Insights
9. Exploratory Data Analysis (EDA)
10. Dashboard Design and Insights
11. Advanced Analytics and AI Modeling
12. Sentiment Analysis
13. Feature Engineering
14. Dataset Merging
15. Train/Test Split
16. Regression Models
17. Classification Models
18. Model Evaluation
19. Tools Research and Selection Effort
20. Project Deployment Effort - Use Case
21. Results
22. Regression Results
23. Classification Results
24. Event Study Results
25. Dashboard Insights
26. Final Interpretation

27.References

Discussion-Ready Code Guide

This version links each important table and figure to the notebook code that created or supports it. In the presentation, do not claim causality. Say: the model tests whether Trump sentiment adds a weak directional signal beyond simple oil baselines.

Abstract

This project examines whether Donald Trump's social media posts can add useful information when analyzing crude oil price movement. The project does not assume that a tweet can explain the oil market by itself. Instead, tweets are treated as one possible signal among many other market drivers, and the work tests whether that signal has measurable value.

The analysis combines Trump posts from the 2017-2025 period with daily crude oil price data and an optional hourly oil dataset for additional context. The workflow includes text cleaning, sentiment analysis, keyword engineering, lag and rolling features, chronological train/test splitting, regression, classification, event-study analysis, and Power BI reporting.

The final merged modeling dataset contained 1,200 daily rows and 228 columns, covering the period from 2017-02-09 to 2023-03-13. The regression results were weak, with the Dummy Mean benchmark producing the best R2 result. The classification results were slightly better, where Sentiment Logistic reached balanced accuracy of 0.5076. Therefore, the project is best presented as a realistic weak-signal BI and machine learning study, not as a high-accuracy trading system.

Acknowledgment

We would like to thank Dr. Ayman Mansour for his supervision, guidance, and feedback throughout this graduation project. We also thank the Department of Business Intelligence and Data Analytics at the University of Petra for providing the academic environment needed to complete this work.

This project required several stages, including data collection, cleaning, modeling, visualization, documentation, and interpretation. The final result reflects both technical work and continuous learning in applied data analytics.

Business Intelligence Project description and objectives

The project studies the relationship between Trump social media sentiment and crude oil price movement using Natural Language Processing, machine learning, and business intelligence dashboards. Crude oil was selected because it affects transportation costs, production expenses, inflation, energy companies, consumer behavior, and supply chain stability.

The business value of the project is not to claim that tweets control oil prices. The value is to show how noisy public communication can be converted into structured indicators, compared with market data, tested using realistic validation, and explained through dashboards.

- Prepare Trump tweets, daily crude oil prices, and optional hourly oil data for analysis.
- Clean tweet text and convert it into sentiment and market-related features.

- Detect oil, energy, geopolitical, trade, sanctions, and inflation language in posts.
- Merge tweet signals with oil market data by date and define next-day prediction targets.
- Build regression and classification models using chronological train/test splitting.
- Compare sentiment-based models against oil-only baselines and dummy benchmarks.
- Design Power BI dashboards to explain the data, insights, and model results clearly.

Code Link - Figure 1 - High-Level Architecture

- Shows the full project pipeline from raw files to dashboard output.
- Connects the conceptual diagram to the actual notebook workflow cells.
- Useful in the defense when explaining the project from end to end.

Notebook excerpt:

```
# Notebook Cell 2
# Run this cell first. If you are using Google Colab, upload the CSV files before running the notebook.

import os
import re
import sys
import math
import warnings
# ... see the notebook for the full cell

# Notebook Cell 25
daily_tweets = df.groupby('date_day').agg(
    avg_sentiment=('sentiment_score', 'mean'),
    avg_polarity=('polarity', 'mean'),
    avg_subjectivity=('subjectivity', 'mean'),
    avg_positive_score=('positive_score', 'mean'),
    avg_neutral_score=('neutral_score', 'mean'),
    avg_negative_score=('negative_score', 'mean'),
# ... see the notebook for the full cell

# Notebook Cell 28
# - oil lag features from days before `t`

daily_merged = pd.merge(daily_tweets, oil_model_daily, on='day', how='inner')
daily_merged = daily_merged.sort_values('day').dropna(subset=['target_next_day_return_pct']).copy()
# Tweet lag features.
tweet_lag_base_cols = [
    'avg_sentiment', 'avg_polarity', 'tweet_count', 'negative_tweets', 'positive_tweets',
# ... see the notebook for the full cell

# Notebook Cell 31
    return train, test

train_df, test_df = chronological_train_test_split(daily_merged, date_col='day', test_size=TEST_SIZE)
print('Train rows:', train_df.shape[0])
print('Test rows:', test_df.shape[0])
print('Train period:', train_df['day'].min(), 'to', train_df['day'].max())
print('Test period:', test_df['day'].min(), 'to', test_df['day'].max())
# ... see the notebook for the full cell

# Notebook Cell 37
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

regression_models = {
    'Dummy Mean': ('baseline', DummyRegressor(strategy='mean')),
    'Oil Baseline Ridge': ('baseline', Ridge(alpha=1.0, random_state=RANDOM_STATE)),
    'Oil Baseline RF': ('baseline', RandomForestRegressor(n_estimators=250, max_depth=5, min_samples_l...
    'Sentiment Ridge': ('sentiment', Ridge(alpha=1.0, random_state=RANDOM_STATE)),
    'Sentiment Huber': ('sentiment', HuberRegressor(max_iter=500)),
# ... see the notebook for the full cell

# Notebook Cell 40
)

classification_models = {
    'Dummy Majority': ('baseline', DummyClassifier(strategy='most_frequent')),
    'Oil Baseline Logistic': ('baseline', LogisticRegression(max_iter=2000, class_weight='balanced', r...
    'Oil Baseline RF': ('baseline', RandomForestClassifier(n_estimators=250, max_depth=5, min_samples_...
    'Sentiment Logistic': ('sentiment', LogisticRegression(max_iter=2000, class_weight='balanced', ran...
    'Sentiment RF': ('sentiment', RandomForestClassifier(n_estimators=250, max_depth=5, min_samples_le...
# ... see the notebook for the full cell
```

Description: These cells support the architecture diagram: setup, aggregation, merge, chronological split, regression, classification, and dashboard output.

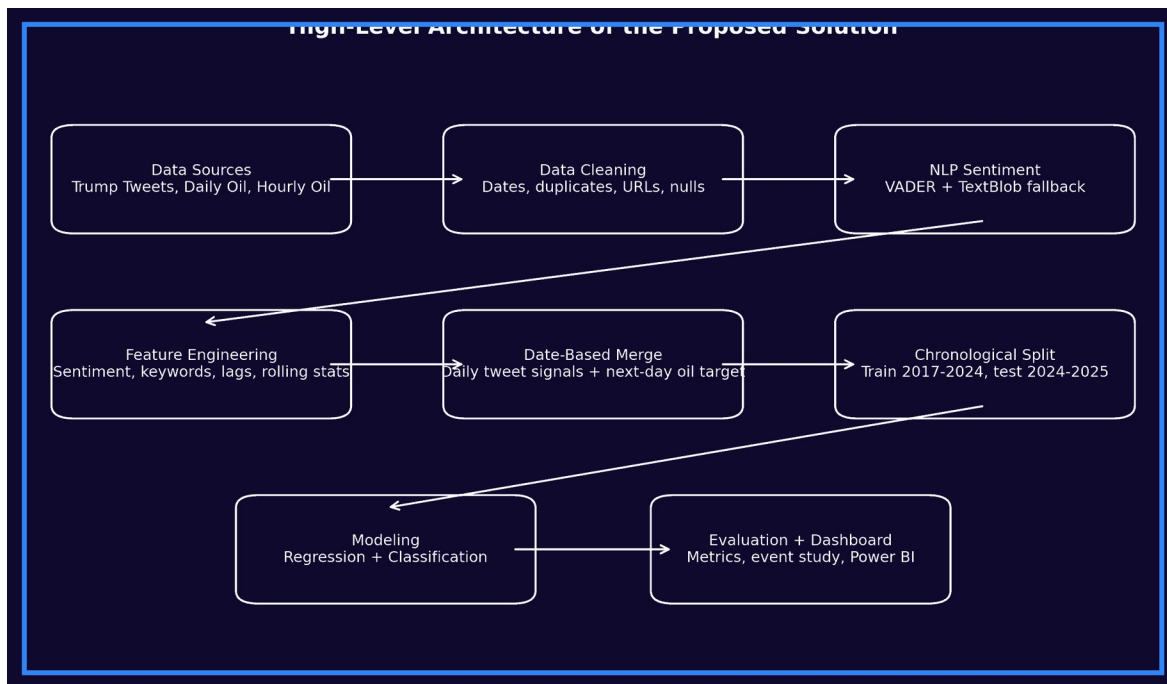


Figure 1. High-Level Architecture of the Proposed Solution

Data Research and Acquiring Effort

The data search focused on public datasets that could support a realistic BI and analytics workflow. The main requirement was to have a large enough collection of Trump posts and a reliable crude oil price series that could be merged by date. Hourly oil data was also reviewed to support short-term market context.

Three sources were selected: a Kaggle dataset for Trump tweets, a Kaggle dataset for daily crude oil prices, and HistData / Free Forex hourly oil data. The selected datasets were suitable because they provided timestamps, text fields, market price fields, and enough historical coverage for feature engineering and model evaluation.

Links to raw data

Code Link - Table - Links to Raw Data

- Defines the expected raw files and searches for them safely.
- Loads Trump posts, daily oil data, and optional hourly oil data.
- Validates required columns before cleaning starts.

Notebook excerpt:

```
# Notebook Cell 5
# 3) File Paths
# The notebook looks for the expected files in Colab, local folders, and `mnt/data`.
# Expected files:
# - `tweets2017-2026.csv`

TWEETS_PATH_CANDIDATES = [
```

```

'/content/tweets2017-2026.csv',
'/content/tweets2017-2026(2).csv',

# Notebook Cell 6
# This cell only loads the files. Cleaning is done in later sections.

df_raw = pd.read_csv(TWEETS_PATH)
oil_daily_raw = pd.read_csv(OIL_DAILY_PATH)
oil_hourly_raw = None
if OIL_HOURLY_PATH is not None:
    oil_hourly_raw = pd.read_csv(OIL_HOURLY_PATH)
print('Tweets shape:', df_raw.shape)

# Notebook Cell 7
# This prevents silent failure when expected columns are missing.

required_tweet_cols = ['date', 'text']
missing_tweet_cols = [c for c in required_tweet_cols if c not in df_raw.columns]
if missing_tweet_cols:
    raise ValueError(f'Missing required tweet columns: {missing_tweet_cols}')
print('Tweet columns available:')
print(df_raw.columns.tolist())
print('\nOil daily columns available:')
# 5) Basic Column Validation

```

Description: This code is the executable version of the raw-data section. It prevents hidden file or column errors before the analysis starts.

Source	Description	Link
Trump Tweets 2009-2025	Public Trump posts dataset used for text cleaning, sentiment analysis, and keyword engineering.	https://www.kaggle.com/datasets/datadrivendecision/trump-tweets-2009-2025
Crude Oil Historical Data Daily Updated	Daily crude oil price data used for return, volatility, lag, and next-day target creation.	https://www.kaggle.com/datasets/isaaclopgu/crude-oil-historical-data-daily-updated
HistData / Free Forex Historical Data	Hourly oil data reviewed for shorter market-reaction context.	https://www.histdata.com/download-free-forex-historical-data/?locale=en

Data Description and understandings

The project uses tweet-level fields, oil market fields, and engineered modeling fields. The original tweet table contains individual posts, while the modeling table is daily. This distinction matters: the model does not predict from individual tweets directly; it predicts from daily aggregated tweet signals merged with daily oil records.

Code Link - Table - Dataset Size and Structure

- Checks actual row and column counts from the loaded files.
- Separates raw tweet data, daily oil data, hourly data, and final merged data.
- Supports the dataset-size table with notebook-based evidence.

Notebook excerpt:

```

# Notebook Cell 6
oil_hourly_raw = pd.read_csv(OIL_HOURLY_PATH)

print('Tweets shape:', df_raw.shape)
print('Daily oil shape:', oil_daily_raw.shape)
if oil_hourly_raw is not None:
    print('Hourly oil shape:', oil_hourly_raw.shape)
display_dark_table(df_raw.head())
display_dark_table(oil_daily_raw.head())

# Notebook Cell 18
# 11) EDA Overview
# EDA can use all available historical data. Modeling evaluation will use only the chronological test ...

```

```

summary_cards = pd.DataFrame({
    'Metric': [
        'Total Posts', 'Start Date', 'End Date', 'Sentiment Engine',
        'Average Polarity', 'Market-Related Posts', 'Direct Oil/Energy Posts'
    ],
    'Value': [
        len(df),
    ]
})

# Notebook Cell 59
if oil_hourly_raw is None:
    print('No hourly oil file found. Skipping hourly analysis.')
else:
    oil_hourly = normalize_colnames(oil_hourly_raw)
    datetime_col = find_col(oil_hourly.columns, ['datetime', 'Date', 'date', 'time'])
    close_h_col = find_col(oil_hourly.columns, ['close', 'Close'])
    oil_type_col = find_col(oil_hourly.columns, ['oil_type', 'type', 'name', 'ticker'])

    if datetime_col is None or close_h_col is None:
        print('Hourly file found, but required datetime/close columns were not detected. Skipping hour...')

# Notebook Cell 61
final_summary = []

final_summary.append(f'Dataset period: {daily_merged["day"].min()} to {daily_merged["day"].max()}')
final_summary.append(f'Sentiment engine used: {SENTIMENT_ENGINE}')
final_summary.append(f'Market-related posts detected: {int(df["is_market_related"].sum())}')
final_summary.append(f'Direct oil/energy posts detected: {int(df["is_oil_direct_related"].sum())}')
final_summary.append(f'Best regression model: {best_reg["Model"]} | R2={best_reg["R2"]:.4f} | MAE={best_reg["MAE"]:.4f}')
final_summary.append(f'Best classification model: {best_clf_name}')

```

Description: This code is important for discussion because dataset sizes should come from the notebook output, not from guessing.

Dataset	Rows	Columns
Trump tweets raw data	32,094	18
Cleaned tweets data	32,094	33
Daily crude oil raw/model data	2,252	8 raw / 44 engineered
Hourly oil raw data	89,395	7
Selected hourly WTI data	39,837	9
Final merged modeling data	1,200	228

Code Link - Table - Field / Feature Group Meaning

- Creates the date field, cleaned text, sentiment variables, keywords, oil returns, volatility, lags, and targets.
- Explains why raw columns became many engineered modeling columns.
- Links every feature group in the table to the actual transformation code.

Notebook excerpt:

```

# Notebook Cell 9

df['weekofyear'] = df['date'].dt.isocalendar().week.astype(int)
df['date_day'] = df['date'].dt.floor('D')
df['date_hour'] = df['date'].dt.floor('H')
df['tweet_length_ch'] = df['text'].astype(str).str.len()
df['tweet_length_class'] = pd.cut(
    df['tweet_length_ch'],
    bins=[-1, 130, 280, np.inf],
    labels=['short', 'medium', 'long']
)
df = df_raw.copy()
df = df.dropna(subset=['text']).copy()
df['date'] = pd.to_datetime(df['date'], errors='coerce', utc=True)
# ... see the notebook for the full cell

# Notebook Cell 10
# Critical fix: remove noisy URL artifacts such as `pic.twitter.com/...`, even when the link appears w...

def clean_text_for_sentiment(text):
    x = str(text)

```

```

x = re.sub(r'https?:\/\/S+', ' ', x, flags=re.IGNORECASE)
x = re.sub(r'www\.\S+', ' ', x, flags=re.IGNORECASE)
x = re.sub(r'pic\.twitter\.com\/\S+', ' ', x, flags=re.IGNORECASE)
x = re.sub(r't\.\co\/\S+', ' ', x, flags=re.IGNORECASE)
x = re.sub(r'@\w+', ' ', x)
x = re.sub(r'#', ' ', x)

# Notebook Cell 13
def sentiment_scores(text):
    text = str(text)
    if SENTIMENT_ENGINE == 'VADER':
        scores = vader_analyzer.polarity_scores(text)
        return pd.Series({
            'polarity': scores['compound'],
            'positive_score': scores['pos'],
            'neutral_score': scores['neu'],
        })

def sentiment_label(score):
    if score > 0.05:
        return 'Positive'
    elif score < -0.05:
        return 'Negative'
    else:
        return 'Neutral'

# ... see the notebook for the full cell

# Notebook Cell 15
market_keywords = [
    'oil', 'gas', 'energy', 'crude', 'crude oil', 'petroleum', 'opec',
    'saudi', 'iran', 'iraq', 'russia', 'china', 'war', 'missile',
    'sanction', 'sanctions', 'tariff', 'tariffs', 'trade', 'supply', 'supply chain',
    'shipping', 'inflation', 'prices', 'market', 'economy', 'economic',
    'jobs', 'dollar', 'fed', 'interest', 'interest rate', 'rates', 'stock', 'stocks',
    'exports', 'imports', 'manufacturing', 'pipeline', 'refinery', 'fuel'
]

# Notebook Cell 27
oil_daily = oil_daily.sort_values('day').drop_duplicates(subset=['day'], keep='last')

oil_daily['oil_return_pct_same_day'] = (oil_daily['oil_close'] - oil_daily['oil_open']) / oil_daily['oil_open']
oil_daily['oil_delta_same_day'] = oil_daily['oil_close'] - oil_daily['oil_open']
oil_daily['oil_volatility_same_day'] = oil_daily['oil_high'] - oil_daily['oil_low']
for lag in [1, 2, 3, 5, 7, 10, 14]:
    oil_daily[f'oil_return_lag{lag}'] = oil_daily['oil_return_pct_same_day'].shift(lag)
    oil_daily[f'oil_close_lag{lag}'] = oil_daily['oil_close'].shift(lag)
    oil_daily[f'oil_volume_lag{lag}'] = oil_daily['oil_volume'].shift(lag)
    oil_daily[f'oil_volatility_lag{lag}'] = oil_daily['oil_volatility_same_day'].shift(lag)

# Notebook Cell 28
daily_merged = pd.merge(daily_tweets, oil_model_daily, on='day', how='inner')
daily_merged = daily_merged.sort_values('day').dropna(subset=['target_next_day_return_pct']).copy()
# Tweet lag features.
tweet_lag_base_cols = [
    'avg_sentiment', 'avg_polarity', 'tweet_count', 'negative_tweets', 'positive_tweets',
    'market_keyword_mentions', 'geo_keyword_mentions', 'oil_direct_keyword_mentions',
    'market_related_posts', 'geo_related_posts', 'oil_direct_related_posts',
    'total_engagement', 'total_final_impact', 'total_market_importance', 'total_oil_direct_importance'
]

```

Description: This block supports the feature explanation table. It shows the main transformations behind the modeling dataset.

Field / Feature Group	Meaning	Importance
Tweet date / day	The date extracted from each post timestamp.	Used to aggregate tweets and merge them with oil records.
Cleaned text	Tweet text after removing links, media artifacts, and unusable symbols.	Improves the quality of NLP sentiment scoring.
Sentiment score	Positive, negative, neutral, and final impact sentiment measures.	Main text signal tested against crude oil movement.
Keyword counts	Market, geopolitical, trade, sanctions, oil, and energy mentions.	Separates general posts from potentially market-relevant posts.
Engagement features	Likes, reposts, and virality-related measures.	Approximates how visible or important a post may have been.
Oil price fields	Open, high, low, close, adjusted close, and volume.	Base market information for oil-only baseline models.
Oil return and volatility	Daily return percentage, absolute movement, and volatility measures.	Used for modeling and comparison with sentiment patterns.

Lag and rolling features	Previous-day and rolling-window versions of sentiment and oil variables.	Avoids relying only on same-day information and supports time-series learning.
Next-day target	Next-day oil return and next-day direction.	Defines the actual prediction task.

Data Primary Cleaning and Transformation

The cleaning and transformation stage was the most important part of the pipeline. The raw tweet data was noisy, while the oil data needed return and volatility calculations before it could be used in modeling. The final dataset was created only after several controlled preparation steps.

1. Converted date columns into consistent datetime format.
2. Created daily date fields so tweets could be grouped by day.
3. Removed URLs, media links, and social media artifacts from tweet text.
4. Handled missing or invalid text values and checked duplicate records.
5. Scored tweet sentiment using VADER, with TextBlob kept as a backup option.
6. Created keyword features for market, geopolitical, trade, sanctions, oil, and energy topics.
7. Aggregated tweet-level features into daily values such as average sentiment, tweet count, positive tweets, negative tweets, engagement, and keyword mentions.
8. Engineered oil features such as daily return, volatility, lags, rolling means, and rolling standard deviations.
9. Merged tweet features and oil features by date.
10. Shifted the target to next-day oil return and next-day direction to reduce future leakage.

The 228 columns in the final modeling dataset are not raw columns. Most of them are engineered features, including sentiment scores, keyword counts, lag features, rolling statistics, and target variables.

Data Visualization and Insights

The visual analysis was divided into notebook exploratory charts and Power BI dashboard pages. The notebook visuals helped verify patterns during development, while the dashboard transformed the final results into a clearer BI presentation layer.

Exploratory Data Analysis (EDA)

Code Link - Figure 2 - Sentiment Distribution

- Counts positive, neutral, and negative posts.
- Builds the sentiment distribution chart.
- Used only as descriptive EDA, not as market proof.

Notebook excerpt:

```
if PLOTLY_OK:
    sentiment_counts = df['sentiment'].value_counts().reset_index()
    sentiment_counts.columns = ['sentiment', 'count']
    fig = px.pie(sentiment_counts, names='sentiment', values='count', title='Sentiment Distribution')
    style_fig(fig, height=450).show()
else:
    df['sentiment'].value_counts().plot(kind='pie', autopct='%1.1f%%', figsize=(6,6), title='Sentiment...')
    plt.ylabel('')
    plt.show()
```

Description: The chart summarizes how tweet sentiment is distributed before any market modeling is performed.

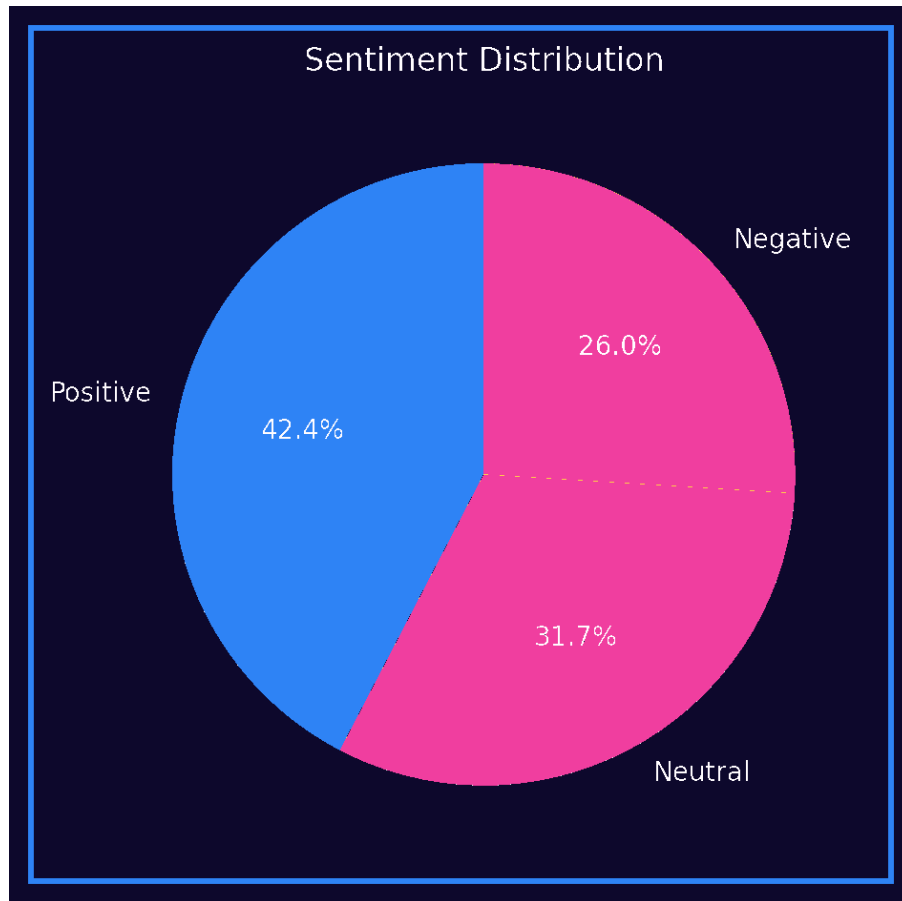


Figure 2. Notebook Sentiment Distribution

The sentiment distribution shows that positive posts were more common than negative posts, with neutral posts also representing a large share. This is useful as a starting point, but sentiment distribution alone does not prove any market effect.

Code Link - Figure 3 - Posts by Year

- Groups tweets by year.
- Counts post volume for each year.
- Plots time-based tweet activity.

Notebook excerpt:

```
posts_by_year = df.groupby('year', as_index=False).agg(posts=('id', 'count'))

if PLOTLY_OK:
    fig = px.line(posts_by_year, x='year', y='posts', markers=True, title='Posts by Year')
    style_fig(fig).show()
else:
    posts_by_year.plot(x='year', y='posts', marker='o', title='Posts by Year')
    plt.show()
```

Description: This figure explains why tweet volume varies across years and why aggregation by date matters.

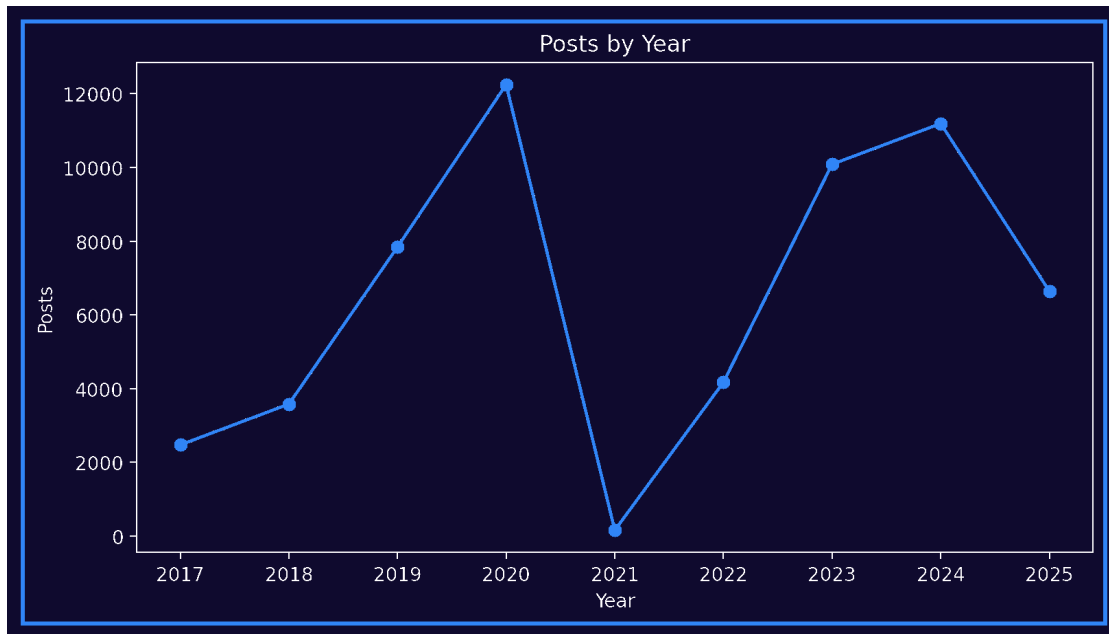


Figure 3. Notebook Posts by Year

Code Link - Figure 4 - Top Platforms

- Counts posts by platform.
- Highlights Twitter vs Truth Social activity.
- Supports later platform-shift interpretation.

Notebook excerpt:

```
platform_counts = df['platform'].replace('', 'Unknown').value_counts().head(10).reset_index()
platform_counts.columns = ['platform', 'posts']

if PLOTLY_OK:
    fig = px.bar(platform_counts, x='platform', y='posts', title='Top Platforms')
    style_fig(fig).show()
else:
    platform_counts.plot(kind='bar', x='platform', y='posts', title='Top Platforms')
    plt.show()
```

Description: This code helps explain that the data source changed over time, which can affect the model signal.

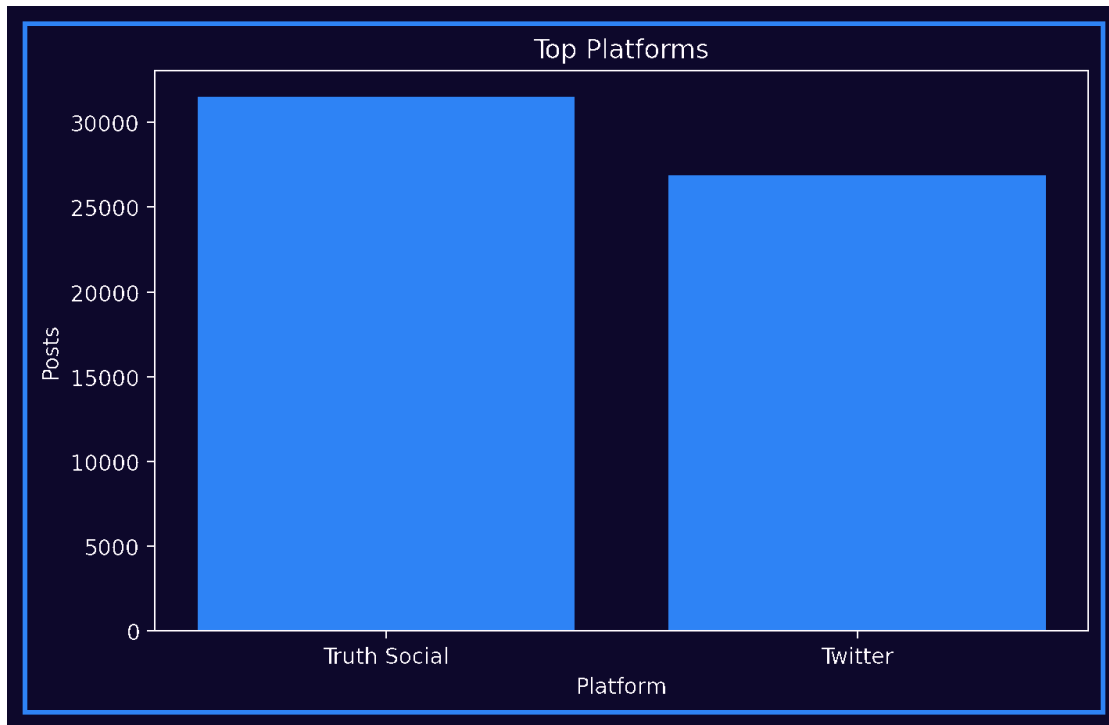


Figure 4. Notebook Top Platforms

Tweet activity changed strongly across years and platforms. This matters because the volume of posts can influence the daily aggregation layer and should not be ignored during interpretation.

Code Link - Figure 5 - Market, Geopolitical, and Oil-Related Posts by Year

- Aggregates market-related, geopolitical, and direct oil posts by year.
- Compares broad market language with direct oil language.
- Justifies using indirect keywords, not only the word oil.

Notebook excerpt:

```
market_by_year = df.groupby('year', as_index=False).agg(
    total_posts=('id', 'count'),
    market_posts=('is_market_related', 'sum'),
    geo_posts=('is_geo_related', 'sum'),
    oil_direct_posts=('is_oil_direct_related', 'sum')
)
market_by_year['market_share'] = market_by_year['market_posts'] / market_by_year['total_posts']

value_name='Posts'
)
fig = px.line(melted, x='year', y='Posts', color='Type', markers=True, title='Market/Geopolitical/...'
style_fig(fig).show()
```

Description: This figure supports the project logic: Trump posts can be market-related even when they do not directly mention oil.

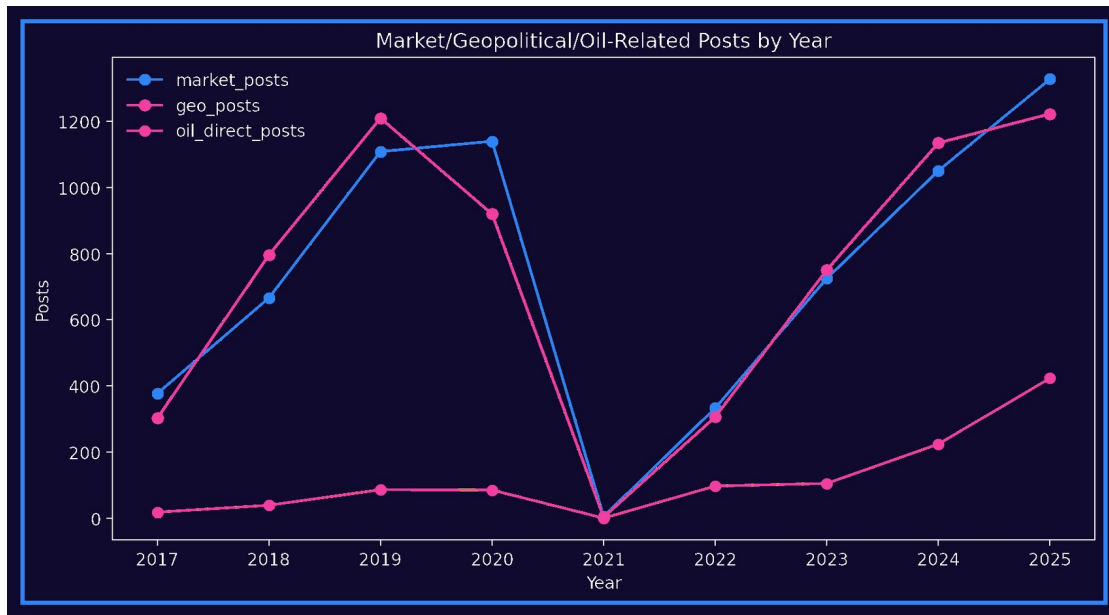


Figure 5. Market, Geopolitical, and Oil-Related Posts by Year

Code Link - Figure 6 - Top Hashtags

- Extracts hashtags from tweet text.
- Counts the most common hashtags.
- Provides descriptive context about tweet themes.

Notebook excerpt:

```
# This is descriptive only and does not enter the model unless explicitly engineered.

all_hashtags = []
for text, hashtags in zip(df['text'], df['hashtags']):
    found = re.findall(r'#\w+', str(text).lower())
    if not found:
        found = re.findall(r'#\w+', str(hashtags).lower())
    all_hashtags.extend(found)
top_hashtags = pd.DataFrame(Counter(all_hashtags).most_common(20), columns=['hashtag', 'count'])
if len(top_hashtags) > 0 and PLOTLY_OK:
    fig = px.bar(top_hashtags, x='count', y='hashtag', orientation='h', title='Top Hashtags')
    style_fig(fig).show()
```

Description: Hashtags are used for EDA and interpretation; they are not treated as causal evidence.

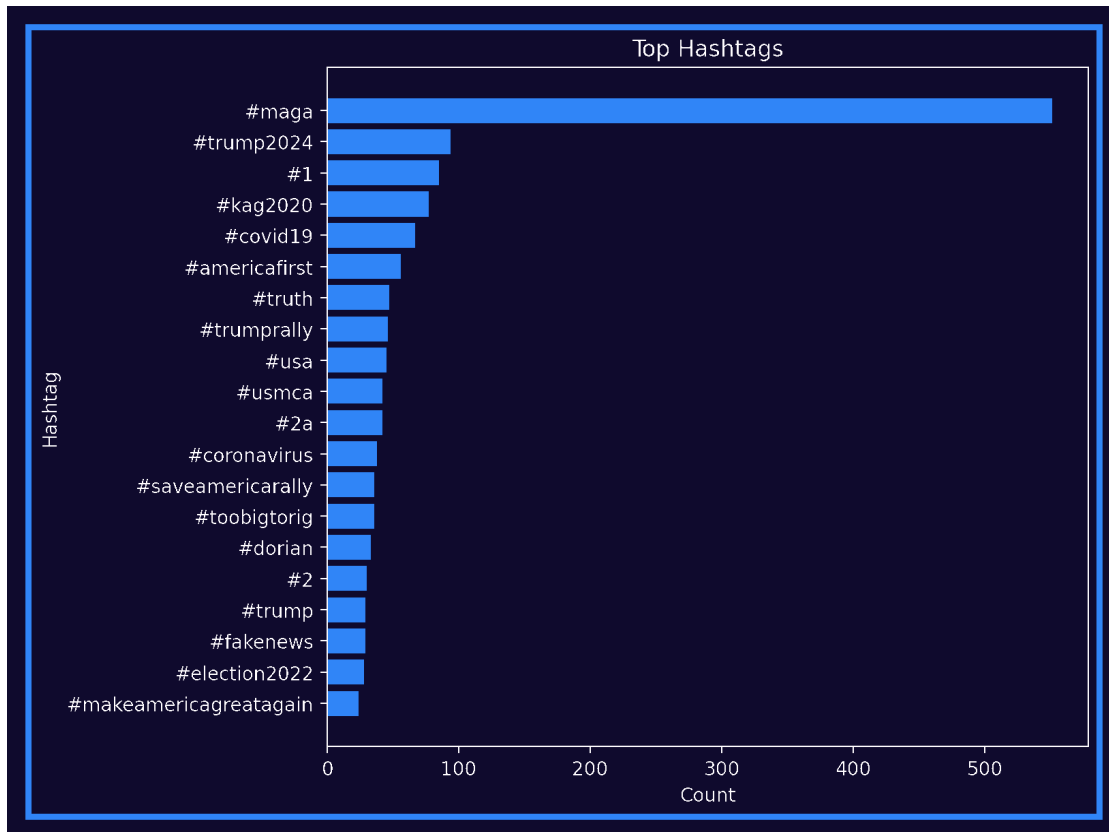


Figure 6. Notebook Top Hashtags

Market and geopolitical posts were more frequent than direct oil-related posts. That supports the decision to include indirect market language such as sanctions, trade, China, Iran, Russia, inflation, and energy terms instead of only searching for the word oil.

Dashboard Design & Insights

Code Link - Figure 7 - Power BI Summary Dashboard

- Builds daily tweet features used by the reporting layer.
- Merges tweet features with next-day oil targets.
- Exports prepared outputs for dashboard refresh.

Notebook excerpt:

```
# Notebook Cell 25
daily_tweets = df.groupby('date_day').agg(
    avg_sentiment=('sentiment_score', 'mean'),
    avg_polarity=('polarity', 'mean'),
    avg_subjectivity=('subjectivity', 'mean'),
    avg_positive_score=('positive_score', 'mean'),
    avg_neutral_score=('neutral_score', 'mean'),
    avg_negative_score=('negative_score', 'mean'),
    tweet_count=('text', 'count'),
    negative_tweets=('sentiment', lambda x: (x == 'Negative').sum()),

# Notebook Cell 28
# - oil lag features from days before `t`

daily_merged = pd.merge(daily_tweets, oil_model_daily, on='day', how='inner')
daily_merged = daily_merged.sort_values('day').dropna(subset=['target_next_day_return_pct']).copy()
# Tweet lag features.
```

```

tweet_lag_base_cols = [
    'avg_sentiment', 'avg_polarity', 'tweet_count', 'negative_tweets', 'positive_tweets',
    'market_keyword_mentions', 'geo_keyword_mentions', 'oil_direct_keyword_mentions',
    'market_related_posts', 'geo_related_posts', 'oil_direct_related_posts',
    'total_engagement', 'total_final_impact', 'total_market_importance', 'total_oil_direct_importance'
]

# Notebook Cell 62
OUTPUT_DIR.mkdir(exist_ok=True)

summary_cards.to_csv(OUTPUT_DIR / 'summary_cards.csv', index=False)
summary_keyword_check.to_csv(OUTPUT_DIR / 'keyword_detection_check.csv', index=False)
reg_results_df.to_csv(OUTPUT_DIR / 'regression_results.csv', index=False)
clf_results_df.to_csv(OUTPUT_DIR / 'classification_results.csv', index=False)
comparison_summary.to_csv(OUTPUT_DIR / 'baseline_comparison.csv', index=False)
corr_df.to_csv(OUTPUT_DIR / 'correlation_with_next_day_return.csv')
test_predictions_df.to_csv(OUTPUT_DIR / 'test_predictions.csv', index=False)

```

Description: This code produces the prepared dataset behind the dashboard KPIs such as total tweets, average sentiment, and next-day return.



Figure 7. Power BI Summary Dashboard: Tweet Volume and Next-Day Oil Return

The Power BI summary page reports around 41K tweets, an average sentiment of 0.21, an average next-day oil return of -0.30%, 11K negative tweets, and 17K positive tweets. These numbers are from the prepared reporting layer, not the original raw tweet file.

Code Link - Figure 8 - Positive/Negative Tweet Distribution and Keyword Mentions

- Creates positive and negative tweet counts.
- Creates market and oil keyword mention fields.
- Supports the Power BI dashboard page comparing sentiment and keyword activity.

Notebook excerpt:

```

# Notebook Cell 25
avg_negative_score=('negative_score', 'mean'),
tweet_count=('text', 'count'),
negative_tweets=('sentiment', lambda x: (x == 'Negative').sum()),
positive_tweets=('sentiment', lambda x: (x == 'Positive').sum()),
neutral_tweets=('sentiment', lambda x: (x == 'Neutral').sum()),
total_likes=('favorite_count', 'sum'),
total_reposts=('repost_count', 'sum'),

```

```

total_engagement=('engagement', 'sum'),
avg_word_count=('word_count', 'mean'),
hashtag_count=('hashtag_count', 'sum'),
mention_count=('mention_count', 'sum'),
market_keyword_mentions=('market_keyword_count', 'sum'),
geo_keyword_mentions=('geo_keyword_count', 'sum'),
# ... see the notebook for the full cell

# Notebook Cell 52

viz_sentiment_oil = daily_merged[
    ['day', 'negative_tweets', 'positive_tweets',
     'negative_ratio', 'positive_ratio',
     'target_next_day_return_pct',
     'market_keyword_mentions', 'oil_direct_keyword_mentions']
].copy()
if PLOTLY_OK:
    fig = go.Figure()

# Notebook Cell 62
OUTPUT_DIR.mkdir(exist_ok=True)

summary_cards.to_csv(OUTPUT_DIR / 'summary_cards.csv', index=False)
summary_keyword_check.to_csv(OUTPUT_DIR / 'keyword_detection_check.csv', index=False)
reg_results_df.to_csv(OUTPUT_DIR / 'regression_results.csv', index=False)
clf_results_df.to_csv(OUTPUT_DIR / 'classification_results.csv', index=False)
comparison_summary.to_csv(OUTPUT_DIR / 'baseline_comparison.csv', index=False)
corr_df.to_csv(OUTPUT_DIR / 'correlation_with_next_day_return.csv')
# ... see the notebook for the full cell

```

Description: This dashboard is based on aggregated daily reporting fields, not raw unprocessed tweets.



Figure 8. Positive/Negative Tweet Distribution and Keyword Mentions

The second dashboard page shows that market keyword mentions were much more frequent than direct oil keyword mentions. This confirms that the project should be interpreted as a broad market-sentiment analysis rather than a direct oil-only tweet study.

Code Link - Figure 9 - Normalized Sentiment, Market Mentions, and Oil Return Trend

- Standardizes variables using z-scores.
- Plots sentiment, market mentions, and next-day oil return on one scale.
- Prevents misleading comparison between variables with different units.

Notebook excerpt:

```
# Sentiment and oil return are not on the same scale. This chart uses z-scores for visual comparison o...

def zscore(s):
    s = pd.Series(s)
    std = s.std()
    if std == 0 or pd.isna(std):
        return s * 0
    return (s - s.mean()) / std
viz_df = daily_merged[['day', 'avg_sentiment', 'target_next_day_return_pct', 'market_keyword_mentions'...
viz_df['avg_sentiment_z'] = zscore(viz_df['avg_sentiment'])
viz_df['next_day_return_z'] = zscore(viz_df['target_next_day_return_pct'])
viz_df['market_mentions_z'] = zscore(viz_df['market_keyword_mentions'])
viz_df['oil_direct_mentions_z'] = zscore(viz_df['oil_direct_keyword_mentions'])
if PLOTLY_OK:
    fig = go.Figure()
```

Description: The code makes the trend comparison visually fair, but the result remains descriptive and non-causal.

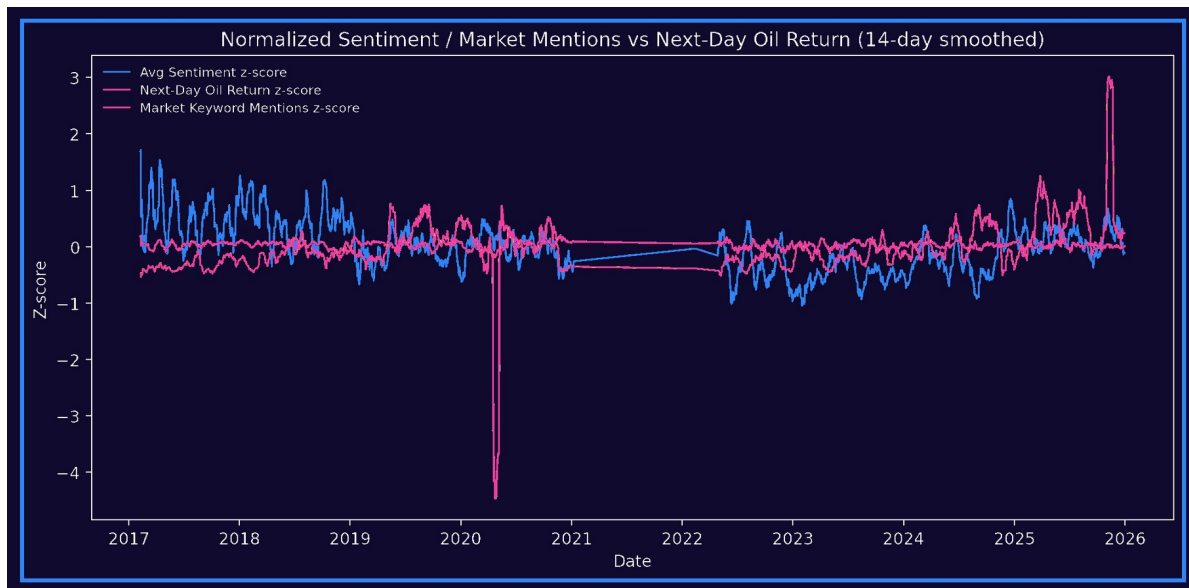


Figure 9. Normalized Sentiment, Market Mentions, and Oil Return Trend

The normalized trend chart helps compare sentiment, market mentions, and next-day oil return on the same scale. The chart shows movement in all three variables, but the relationship is not stable enough to support a strong causal claim.

Advanced Analytics and AI Modeling

The modeling stage was designed to test whether sentiment-based features add value beyond simple benchmarks. Two modeling tasks were used: regression for the size of next-day oil return and classification for next-day direction.

- Regression models: Dummy Mean, Oil Baseline Ridge, Oil Baseline Random Forest, Sentiment Ridge, Sentiment Huber, Sentiment Random Forest, and Sentiment Gradient Boosting.
- Classification models: Dummy Majority, Oil Baseline Logistic, Oil Baseline Random Forest, Sentiment Logistic, Sentiment Random Forest, and Sentiment Gradient Boosting.
- Evaluation metrics for regression: MAE, RMSE, and R2.

- Evaluation metrics for classification: accuracy, balanced accuracy, precision, recall, F1 score, ROC AUC, classification reports, and confusion matrices.
- Train/test split: chronological split, with 960 training rows from 2017-02-09 to 2020-12-07 and 240 testing rows from 2020-12-08 to 2023-03-13.

Code Link - Table - Chronological Train/Test Split

- Splits by time order, not randomly.
- Uses older rows for training and newer rows for testing.
- Checks distribution shift after the split.

Notebook excerpt:

```
# Notebook Cell 31
return train, test

train_df, test_df = chronological_train_test_split(daily_merged, date_col='day', test_size=TEST_SIZE)
print('Train rows:', train_df.shape[0])
print('Test rows:', test_df.shape[0])
print('Train period:', train_df['day'].min(), 'to', train_df['day'].max())
print('Test period:', test_df['day'].min(), 'to', test_df['day'].max())
print('Chronological split check:', train_df['day'].max() < test_df['day'].min())

# Notebook Cell 32
train_platform = platform_pivot[platform_pivot['day'].isin(train_df['day'])]['dominant_platform'].valu...
test_platform = platform_pivot[platform_pivot['day'].isin(test_df['day'])]['dominant_platform'].value...
platform_shift = pd.concat([train_platform, test_platform], axis=1).fillna(0)

display_dark_table(platform_shift)
# This matters because Trump moved from Twitter to Truth Social. If the platform distribution changes, ...
platform_daily = df.groupby(['date_day', 'platform']).size().reset_index(name='posts')
platform_pivot = platform_daily.pivot_table(index='date_day', columns='platform', values='posts', fill...
platform_pivot = platform_pivot.rename(columns={'date_day': 'day'})
platform_cols = [c for c in platform_pivot.columns if c != 'day']
# ... see the notebook for the full cell
```

Description: This is one of the most important defense points: random split would leak future information and exaggerate performance.

Subset	Rows	Period
Training set	960	2017-02-09 to 2020-12-07
Testing set	240	2020-12-08 to 2023-03-13

The chronological split is essential because market data has time order. A random split could mix future periods into the training set and make the model look stronger than it would be in a real forecasting situation.

Tools Research and Selection Effort

Code Link - Table - Tools Research and Selection

- Shows the imported Python libraries and notebook environment.
- Links NLP, modeling, and visualization tools to their actual usage.
- Connects the tools table to executable code.

Notebook excerpt:

```
# Notebook Cell 2
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
pd.set_option('display.max_columns', 160)
pd.set_option('display.max_colwidth', 240)
RANDOM_STATE = 42
TEST_SIZE = 0.20
warnings.filterwarnings('ignore')

# Notebook Cell 12

try:
    from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
    vader_analyzer = SentimentIntensityAnalyzer()
    SENTIMENT_ENGINE = 'VADER'
except Exception:
    try:
        import subprocess
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'vaderSentiment', '-q'])
        from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
    except Exception:
        vader_analyzer = None
# ... see the notebook for the full cell

# Notebook Cell 4
try:
    import plotly.express as px
    import plotly.graph_objects as go
    import plotly.io as pio
    PLOTLY_OK = True
except Exception:
    PLOTLY_OK = False
    px = None
    go = None
    px.defaults.color_continuous_scale = DASH_CONTINUOUS

def style_fig(fig, title=None, height=520):
    # ... see the notebook for the full cell

```

Description: This table is supported by the packages used throughout the notebook, including pandas, NumPy, VADER/TextBlob, scikit-learn, Plotly, and Matplotlib.

Tool	Purpose in the Project
Google Colab / Jupyter Notebook	Developing the Python workflow, cleaning data, building features, training models, and generating notebook charts.
Python	Main programming language for data analysis and modeling.
Pandas and NumPy	Data loading, cleaning, aggregation, merging, feature engineering, and numerical calculations.
VADER and TextBlob	Sentiment scoring for short social media text.
Scikit-learn	Regression, classification, train/test split, model evaluation, and feature importance.
Matplotlib / Plotly	Notebook charts and exploratory visuals.
Power BI	Interactive dashboard design and business presentation of KPIs, charts, and model results.
Microsoft Word	Final documentation and academic reporting.

These tools were selected because they support the full BI/DA workflow: data acquisition, cleaning, NLP, modeling, visualization, interpretation, and documentation.

Project Deployment Effort - Use Case

A business user would consume this project mainly through the Power BI dashboard and the final analytical report. The dashboard can be used by business intelligence analysts, energy market researchers, and supply chain analysts to review tweet volume, sentiment behavior, keyword activity, event windows, and model performance.

The most realistic deployment is a scheduled analytical dashboard rather than a trading system. In a future version, the workflow could be automated by refreshing tweet data and oil prices, recalculating sentiment features, updating model outputs, and publishing the refreshed Power BI dashboard to business users.

11. User opens the Power BI dashboard.
12. User reviews KPI cards for tweet count, average sentiment, next-day oil return, positive tweets, and negative tweets.
13. User filters by year or keyword type to inspect market-related activity.
14. User reviews model performance to understand whether sentiment improved prediction.
15. User reads the interpretation carefully, especially the limitation that the signal is weak and not causal.

Results

Regression Results

Code Link - Table - Regression Results

- Trains regression models for next-day oil return percentage.
- Calculates MAE, RMSE, and R2.
- Stores results in a comparison table.

Notebook excerpt:

```
}  
reg_results = []  
reg_predictions = {}  
for name, (feature_type, model) in regression_models.items():  
    if feature_type == 'baseline':  
        model.fit(Xb_train_scaled, y_reg_train)  
        pred = model.predict(Xb_test_scaled)  
    else:  
        {}  
reg_results_df = pd.DataFrame(reg_results).sort_values('R2', ascending=False)  
display_dark_table(reg_results_df)
```

Description: This code is the direct source for the regression results table and helps explain why regression performance was weak.

Model	MAE	RMSE	R2
Dummy Mean	2.0340	2.5364	-0.0205
Sentiment Huber	2.1652	2.7600	-0.2083
Sentiment GB	2.2574	2.8516	-0.2898
Oil Baseline RF	3.2215	4.2607	-1.8794
Sentiment RF	3.3899	4.4734	-2.1741
Oil Baseline Ridge	6.1722	7.8063	-8.6658
Sentiment Ridge	6.4409	8.2840	-9.8851

Code Link - Figure 10 - Regression R2 Scores

- Uses regression result metrics.
- Highlights R2 values across models.
- Shows that dummy/average performance was difficult to beat.

Notebook excerpt:

```
# Notebook Cell 37  
{})  
reg_results_df = pd.DataFrame(reg_results).sort_values('R2', ascending=False)  
display_dark_table(reg_results_df)  
'MAE': mean_absolute_error(y_reg_test, pred),  
'RMSE': rmse,  
'R2': r2_score(y_reg_test, pred)  
  
# Notebook Cell 38  
best_reg = reg_results_df.iloc[0]  
print('Best regression model:', best_reg['Model'])
```

```

print('Best R2:', round(best_reg['R2'], 4))
print('Best MAE:', round(best_reg['MAE'], 4))

if best_reg['R2'] < 0:
    print('\nInterpretation: Regression performance is weak. The model does not explain next-day oil r...')
elif best_reg['R2'] < 0.1:
    print('\nInterpretation: Regression signal exists but is weak. Do not overclaim predictive power.')
else:

```

Description: The chart visually confirms that complex models did not strongly improve next-day return-size prediction.

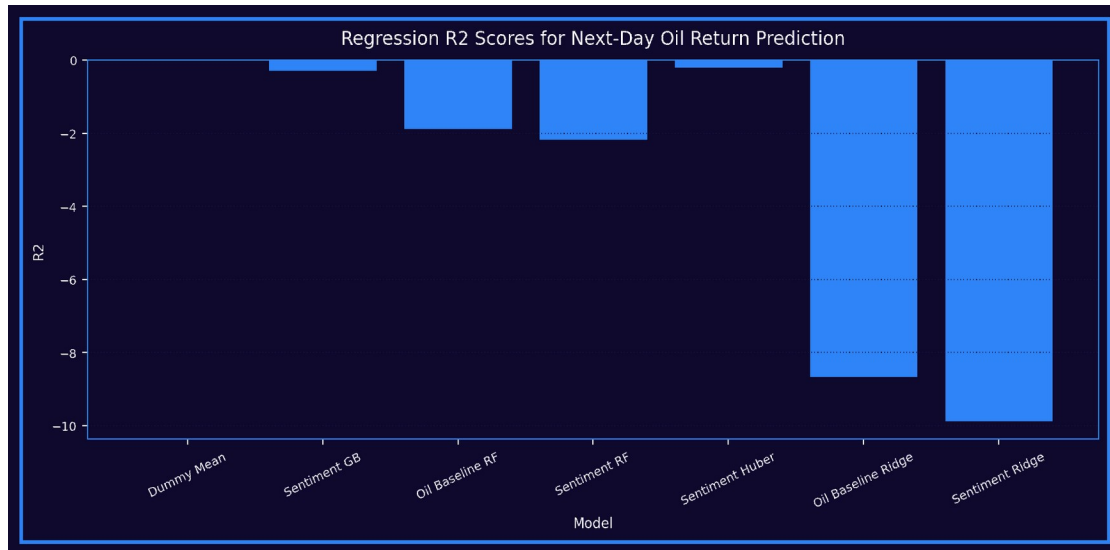


Figure 10. Regression R2 Scores

Code Link - Figure 11 - Notebook Regression Model Comparison

- Builds the notebook regression comparison chart.
- Compares models on R2.
- Supports cautious interpretation of regression results.

Notebook excerpt:

```

if PLOTLY_OK:
    fig = px.bar(reg_results_df, x='Model', y='R2', text='R2', title='Regression Model Comparison - R²')
    fig.update_traces(texttemplate='%{text:.3f}', textposition='outside')
    style_fig(fig, height=520).show()
else:
    reg_results_df.plot(kind='bar', x='Model', y='R2', title='Regression Model Comparison - R²')
    plt.xticks(rotation=45)
    plt.show()
best_reg = reg_results_df.iloc[0]
print('Best regression model:', best_reg['Model'])
print('Best R2:', round(best_reg['R2'], 4))
print('Best MAE:', round(best_reg['MAE'], 4))
if best_reg['R2'] < 0:
    print('\nInterpretation: Regression performance is weak. The model does not explain next-day oil r...')
elif best_reg['R2'] < 0.1:
    print('\nInterpretation: Regression signal exists but is weak. Do not overclaim predictive power.')
# 22) Regression Interpretation Guardrail
# ... see the notebook for the full cell

```

Description: This visual is useful in discussion because weak R2 is not a failure if it is explained honestly.

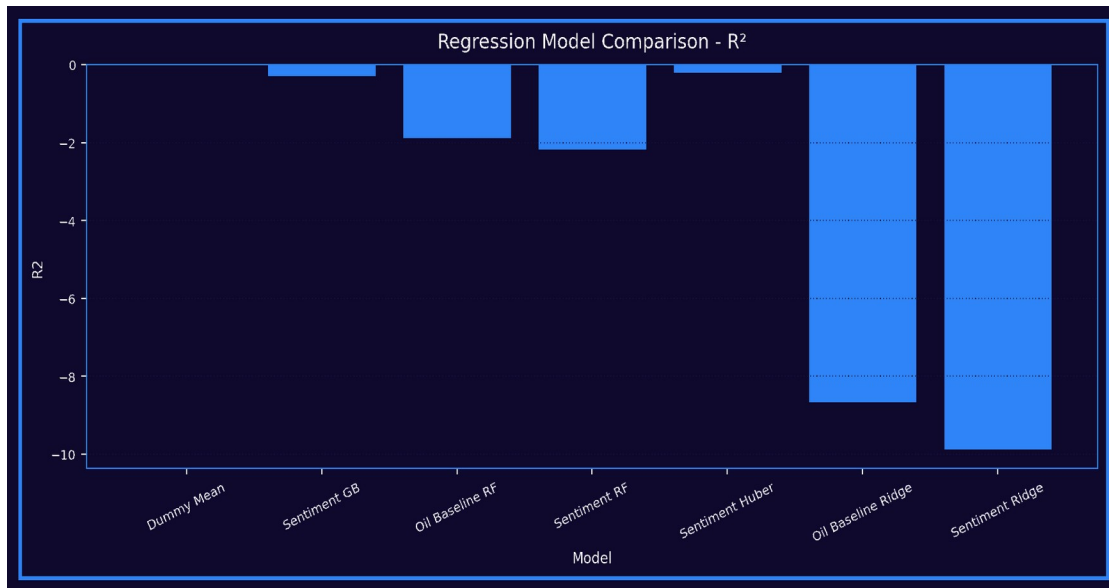


Figure 11. Notebook Regression Model Comparison

The strongest regression result was the Dummy Mean benchmark, with $R^2 = -0.0205$ and $MAE = 2.0340$. This is not an exciting result, but it is important. It shows that the more complex models did not predict the size of next-day oil returns better than a simple average-based approach.

Classification Results

Code Link - Table - Classification Results

- Trains direction models for next-day up/down movement.
- Calculates accuracy, balanced accuracy, precision, recall, F1, and ROC AUC.
- Creates the classification comparison table.

Notebook excerpt:

```
}
clf_results = []
clf_predictions = {}
clf_probabilities = {}
for name, (feature_type, model) in classification_models.items():
    if feature_type == 'baseline':
        model.fit(Xb_train_scaled, y_clf_train)
        pred = model.predict(Xb_test_scaled)
    })
clf_results_df = pd.DataFrame(clf_results).sort_values('Balanced Accuracy', ascending=False)
display_dark_table(clf_results_df)
```

Description: This code is the direct source for the classification table and shows the sentiment model only slightly improved over baseline.

Model	Accuracy	Balanced Acc.	Precision	Recall	F1	ROC AUC
Sentiment Logistic	0.5125	0.5076	0.5131	0.8033	0.6262	0.4803
Dummy Majority	0.5083	0.5000	0.5083	1.0000	0.6740	0.5000
Sentiment GB	0.4917	0.4910	0.5000	0.5328	0.5159	0.4920
Sentiment RF	0.4958	0.4888	0.5023	0.9098	0.6472	0.4654
Oil Baseline RF	0.4750	0.4757	0.4818	0.4344	0.4569	0.4778

Oil Baseline Logistic	0.4417	0.4430	0.4400	0.3607	0.3964	0.4353
-----------------------	--------	--------	--------	--------	--------	--------

Code Link - Figure 12 - Classification Model Performance Comparison

- Reshapes classification metrics.
- Builds the grouped performance chart.
- Compares balanced accuracy, F1, and ROC AUC.

Notebook excerpt:

```
if PLOTLY_OK:
    metric_cols = ['Accuracy', 'Balanced Accuracy', 'F1', 'ROC AUC']
    melted = clf_results_df.melt(id_vars='Model', value_vars=metric_cols, var_name='Metric', value_name='Score')
    fig = px.bar(melted, x='Model', y='Score', color='Metric', barmode='group', title='Classification ...')
    style_fig(fig, height=600).show()
else:
    clf_results_df.plot(kind='bar', x='Model', y=['Accuracy', 'Balanced Accuracy', 'F1'], title='Class...')
    plt.xticks(rotation=45)
    plt.show()
```

Description: The figure makes model comparison easier for non-technical readers.

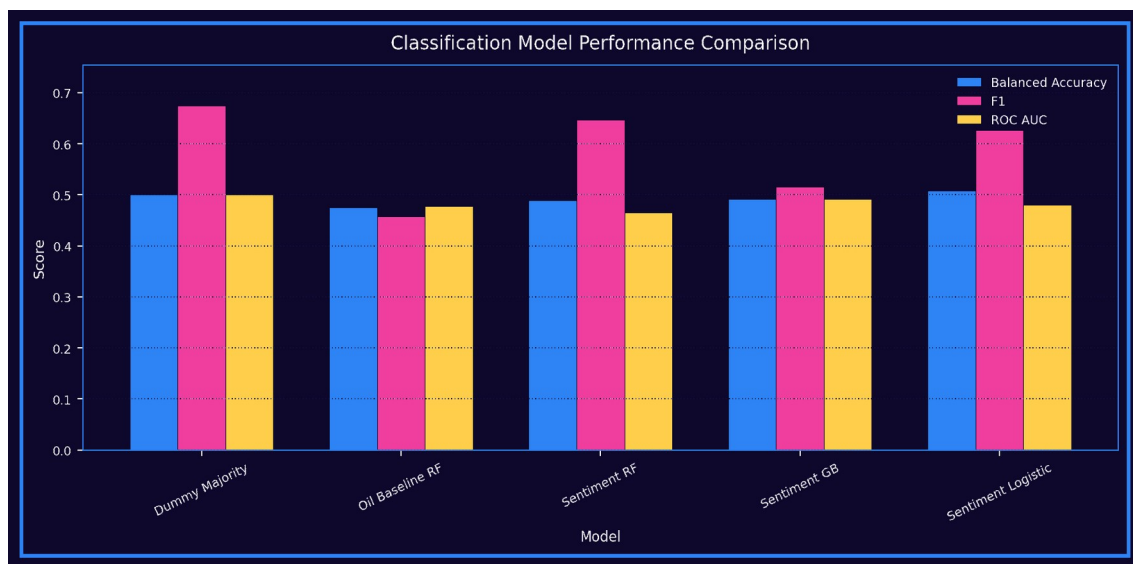


Figure 12. Classification Model Performance Comparison

Code Link - Figure 13 - Notebook Classification Model Comparison

- Uses the same classification metrics.
- Displays notebook-level comparison across models.
- Supports selection of the best classifier.

Notebook excerpt:

```
if PLOTLY_OK:
    metric_cols = ['Accuracy', 'Balanced Accuracy', 'F1', 'ROC AUC']
    melted = clf_results_df.melt(id_vars='Model', value_vars=metric_cols, var_name='Metric', value_name='Score')
    fig = px.bar(melted, x='Model', y='Score', color='Metric', barmode='group', title='Classification ...')
    style_fig(fig, height=600).show()
```



```

else:
    clf_results_df.plot(kind='bar', x='Model', y=['Accuracy', 'Balanced Accuracy', 'F1'], title='Class...
    plt.xticks(rotation=45)
    plt.show()

```

Description: This chart supports the discussion of why Sentiment Logistic was selected as the best directional model.

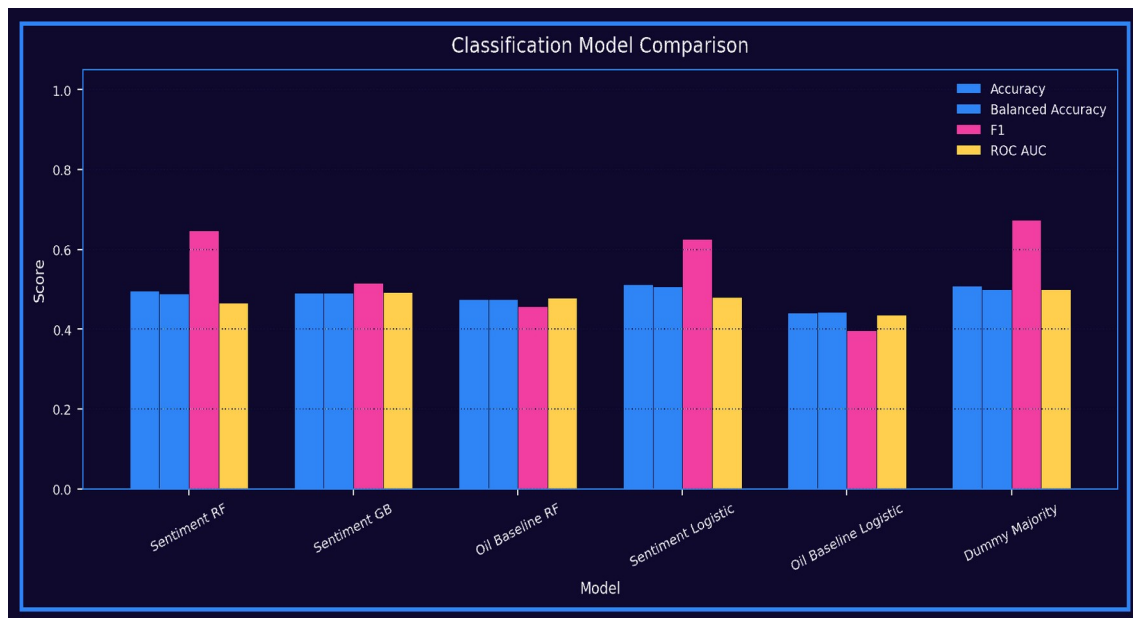


Figure 13. Notebook Classification Model Comparison

Code Link - Figure 14 - Confusion Matrix for Sentiment Logistic

- Reads predictions from the best classifier.
- Creates a classification report.
- Builds the confusion matrix.

Notebook excerpt:

```

print('Best classification model:', best_clf_name)
print('\nClassification report:')
print(classification_report(y_clf_test, best_clf_pred, target_names=['Down/Flat', 'Up'], zero_division...

cm = confusion_matrix(y_clf_test, best_clf_pred)
cm_df = pd.DataFrame(cm, index=['Actual Down/Flat', 'Actual Up'], columns=['Pred Down/Flat', 'Pred Up'])
display_html_table(cm_df)
if PLOTLY_OK:
    fig = px.imshow(cm_df, text_auto=True, title=f'Confusion Matrix - {best_clf_name}')
    style_fig(fig, height=500).show()

```

Description: The confusion matrix is critical because it shows the model still makes many mistakes despite slight improvement.

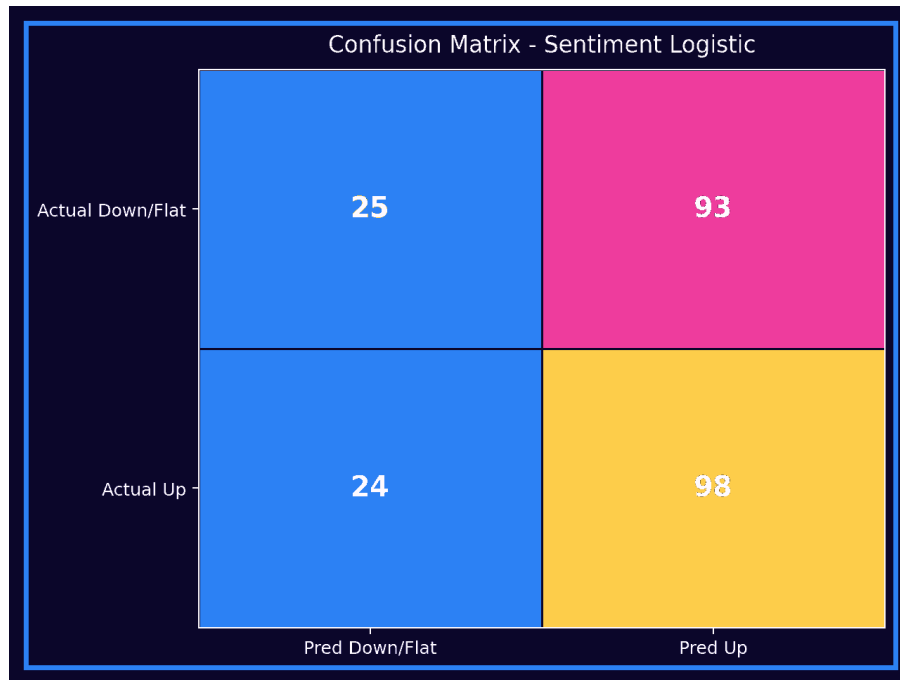


Figure 14. Confusion Matrix for Sentiment Logistic

For classification, the best model was Sentiment Logistic, with accuracy of 0.5125 and balanced accuracy of 0.5076. The best oil-only baseline had balanced accuracy of 0.4757, so the sentiment model improved it by about 0.0319. That improvement exists, but it is small.

The confusion matrix supports the cautious interpretation. The model correctly predicted 25 down/flat days and 98 up days. At the same time, it incorrectly predicted 93 down/flat days as up and 24 up days as down/flat. The model is slightly better than chance, but it still makes many errors.

Event Study and Feature Insights

Code Link - Figure 15 - Oil Return Around Sentiment Shock Days

- Defines positive and negative sentiment shock thresholds.
- Builds event windows from -3 to +3 days.
- Plots average next-day return around shock days.

Notebook excerpt:

```
# Define strong sentiment events using the 10th and 90th percentiles.
negative_event_threshold = event_df['avg_sentiment'].quantile(0.10)
positive_event_threshold = event_df['avg_sentiment'].quantile(0.90)
negative_event_days = event_df.loc[
    event_df['avg_sentiment'] <= negative_event_threshold, 'day'
].tolist()
positive_event_days = event_df.loc[
    event_df['avg_sentiment'] >= positive_event_threshold, 'day'
].tolist()
def build_event_window(data, event_days, label, window=3):
    rows = []
    for event_day in event_days:
        event_index = data.index[data['day'] == event_day]
        if len(event_index) == 0:
            continue
    if PLOTLY_OK:
        fig = px.line(
            event_summary,
            x='relative_day',
```

```

y='avg_next_day_return',
color='event_type',
markers=True,
title='Event Study: Average Next-Day Oil Return Around Sentiment Shocks'
)

```

Description: This event study is descriptive. It helps explore behavior around sentiment shocks but does not prove causality.

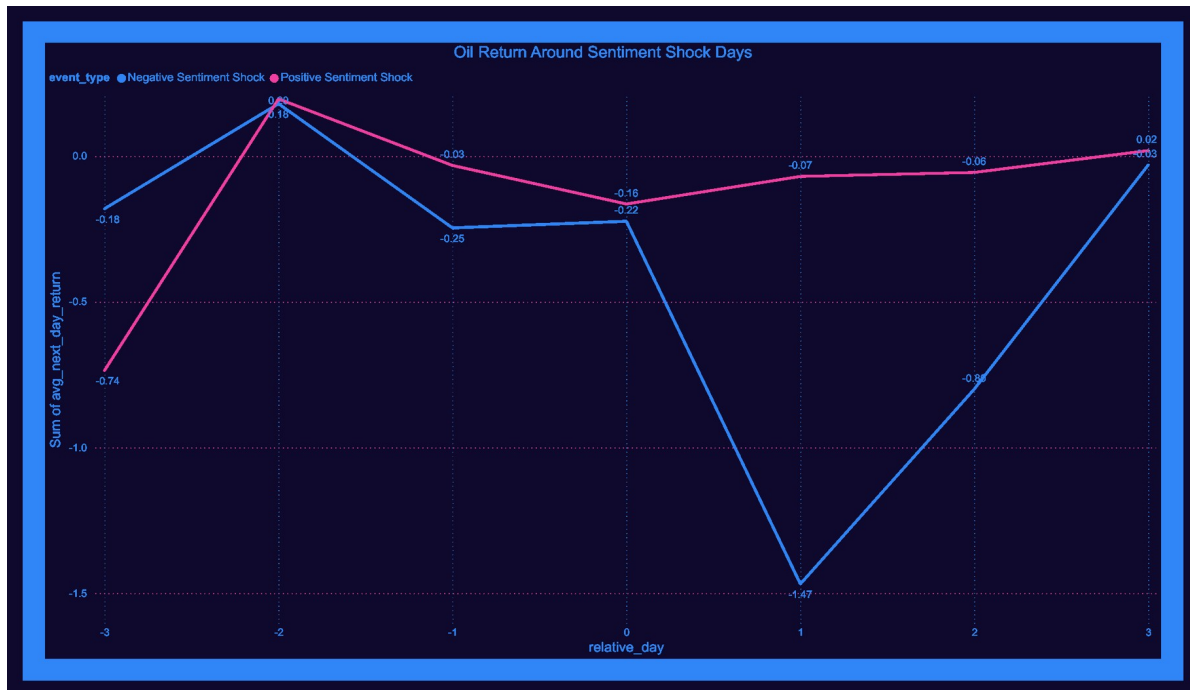


Figure 15. Oil Return Around Sentiment Shock Days

The event-study dashboard examines oil returns around positive and negative sentiment shock days using a -3 to +3 day window. Negative sentiment shocks showed a sharp negative value around the event day, while positive sentiment shocks were less consistent. This is useful descriptive evidence, but it is not proof that tweets caused the movement.

Code Link - Figure 16 - Correlation with Next-Day Oil Return

- Selects sentiment, keyword, and engagement features.
- Calculates correlation with next-day oil return.
- Plots ranked correlations.

Notebook excerpt:

```

corr_cols = [
    'target_next_day_return_pct',
    'avg_sentiment', 'avg_polarity', 'tweet_count', 'negative_tweets', 'positive_tweets',
    'market_keyword_mentions', 'geo_keyword_mentions', 'oil_direct_keyword_mentions',
    'market_related_posts', 'geo_related_posts', 'oil_direct_related_posts',
    'total_engagement', 'avg_final_impact', 'total_final_impact',
    'total_market_importance', 'total_oil_direct_importance'
]
corr_cols = [c for c in corr_cols if c in daily_merged.columns]
plot_corr = corr_df.drop(index='target_next_day_return_pct', errors='ignore').reset_index()
plot_corr.columns = ['Feature', 'Correlation']
fig = px.bar(plot_corr, x='Correlation', y='Feature', orientation='h', title='Correlation with Nex...
style_fig(fig, height=650).show()

```

Description: This figure helps identify weak relationships, but correlation is not causation.

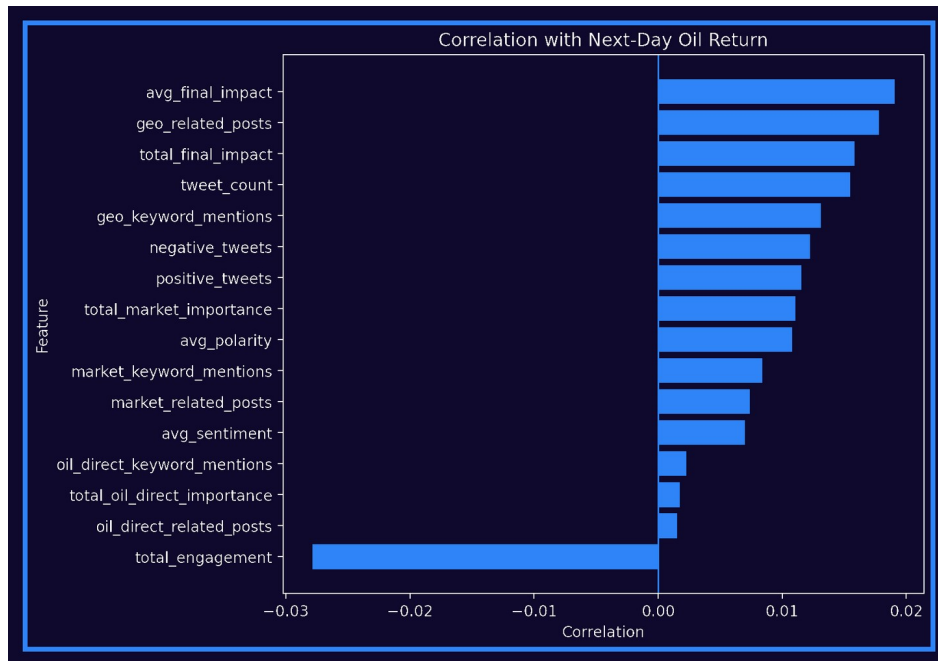


Figure 16. Correlation with Next-Day Oil Return

Code Link - Figure 17 - Top Feature Importances

- Extracts feature importances from the tree-based sentiment model.
- Ranks the most used predictors.
- Plots top feature importance values.

Notebook excerpt:

```
tree_model.fit(Xs_train_scaled, y_clf_train)

if hasattr(tree_model, 'feature_importances_'):
    importance_df = pd.DataFrame({
        'Feature': sentiment_features,
        'Importance': tree_model.feature_importances_
    }).sort_values('Importance', ascending=False).head(25)
    display_dark_table(importance_df)
    if PLOTLY_OK:
        fig = px.bar(importance_df, x='Importance', y='Feature', orientation='h', title=f'Top Feat...
        style_fig(fig, height=700).show()
```

Description: Feature importance explains model behavior only; it does not prove that a feature caused oil movement.

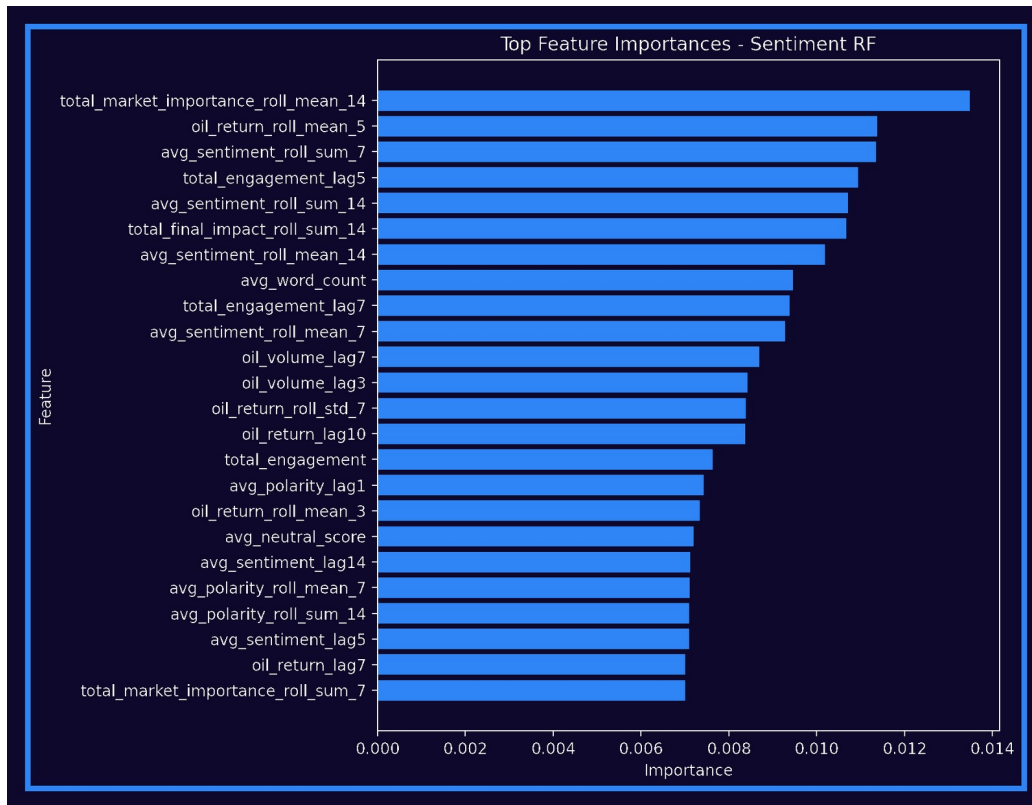


Figure 17. Top Feature Importances for Sentiment Random Forest

Code Link - Figure 18 - Correct vs Incorrect Predictions Over Time

- Builds a test-set prediction table.
- Marks predictions as correct or incorrect.
- Plots errors over time.

Notebook excerpt:

```
# This helps visually inspect when the model gets direction right or wrong.

test_predictions_df = test_df[['day', 'target_next_day', 'target_next_day_return_pct', 'target_next_da...
test_predictions_df['predicted_direction'] = best_clf_pred
test_predictions_df['correct'] = (test_predictions_df['predicted_direction'] == test_predictions_df['t...
display_dark_table(test_predictions_df.head(20))
if PLOTLY_OK:
    plot_df = test_predictions_df.copy()
    plot_df['Actual'] = plot_df['target_next_day_direction'].map({0: 'Down/Flat', 1: 'Up'})
    plot_df['Predicted'] = plot_df['predicted_direction'].map({0: 'Down/Flat', 1: 'Up'})
    fig = px.scatter(
        plot_df,
        x='day',
        y='target_next_day_return_pct',
        color='correct',
        hover_data=['Actual', 'Predicted'],
        title='Best Classifier: Correct vs Incorrect Predictions Over Time'
    )
```

Description: This figure is useful for defense because it shows where the model fails, not just where it works.

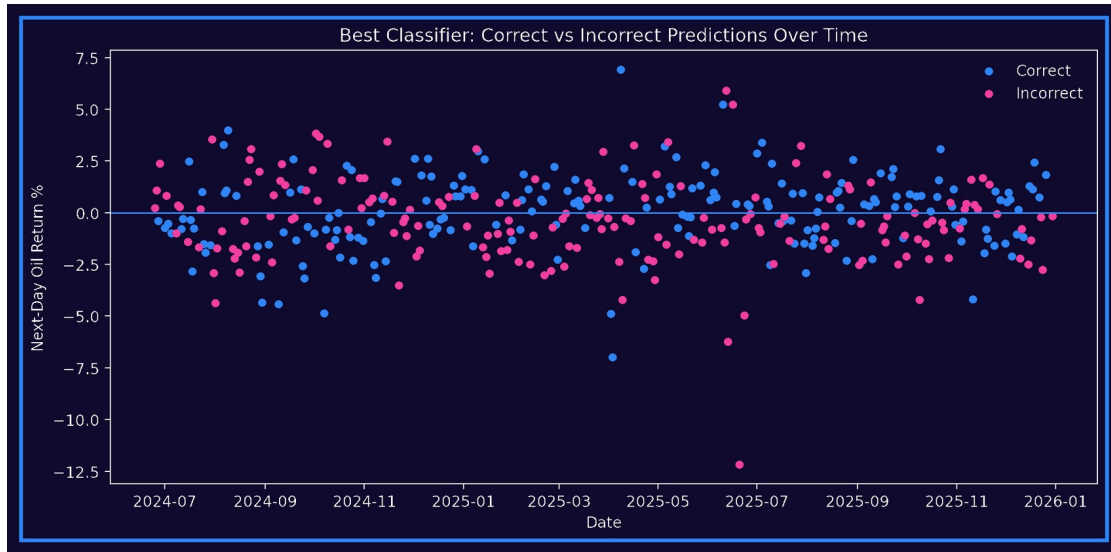


Figure 18. Correct vs Incorrect Predictions Over Time

Code Link - Table - Market Activity Bucket Results

- Creates low, medium, and high market-mention groups.
- Calculates accuracy inside each group.
- Compares average market and oil mentions.

Notebook excerpt:

```
)
bucket_perf = error_analysis_df.groupby('market_activity_bucket', observed=True).agg(
    rows=('correct', 'count'),
    accuracy=('correct', 'mean'),
    avg_market_mentions=('market_keyword_mentions', 'mean'),
    avg_oil_direct_mentions=('oil_direct_keyword_mentions', 'mean')
).reset_index()
display_dark_table(bucket_perf)
error_analysis_df['market_activity_bucket'] = pd.qcut(
    error_analysis_df['market_keyword_mentions'].rank(method='first'),
    q=3,
    labels=['Low Market Mentions', 'Medium Market Mentions', 'High Market Mentions']
)
if PLOTLY_OK:
```

Description: The table shows that more keyword activity does not automatically mean better prediction.

Market Activity Bucket	Rows	Accuracy	Avg Market Mentions	Avg Oil Mentions
Low Market Mentions	80	0.4000	0.0125	0.0000
Medium Market Mentions	80	0.6125	1.6375	0.1500
High Market Mentions	80	0.5250	6.1375	0.8875

Code Link - Figure 19 - Prediction Accuracy by Market-Mention Activity

- Uses the market-activity bucket table.
- Plots accuracy by activity bucket.
- Shows performance differences across low, medium, and high activity.

Notebook excerpt:

```
)  
  
bucket_perf = error_analysis_df.groupby('market_activity_bucket', observed=True).agg(  
    rows=('correct', 'count'),  
    accuracy=('correct', 'mean'),  
    avg_market_mentions=('market_keyword_mentions', 'mean'),  
    avg_oil_direct_mentions=('oil_direct_keyword_mentions', 'mean')  
)  
.reset_index()  
display_dark_table(bucket_perf)  
if PLOTLY_OK:  
    fig = px.bar(bucket_perf, x='market_activity_bucket', y='accuracy', text='accuracy', title='Predic...  
    fig.update_traces(texttemplate='%{text:.3f}', textposition='outside')  
    style_fig(fig, height=500).show()
```

Description: This visualization turns the error-analysis table into a clear dashboard-style insight.

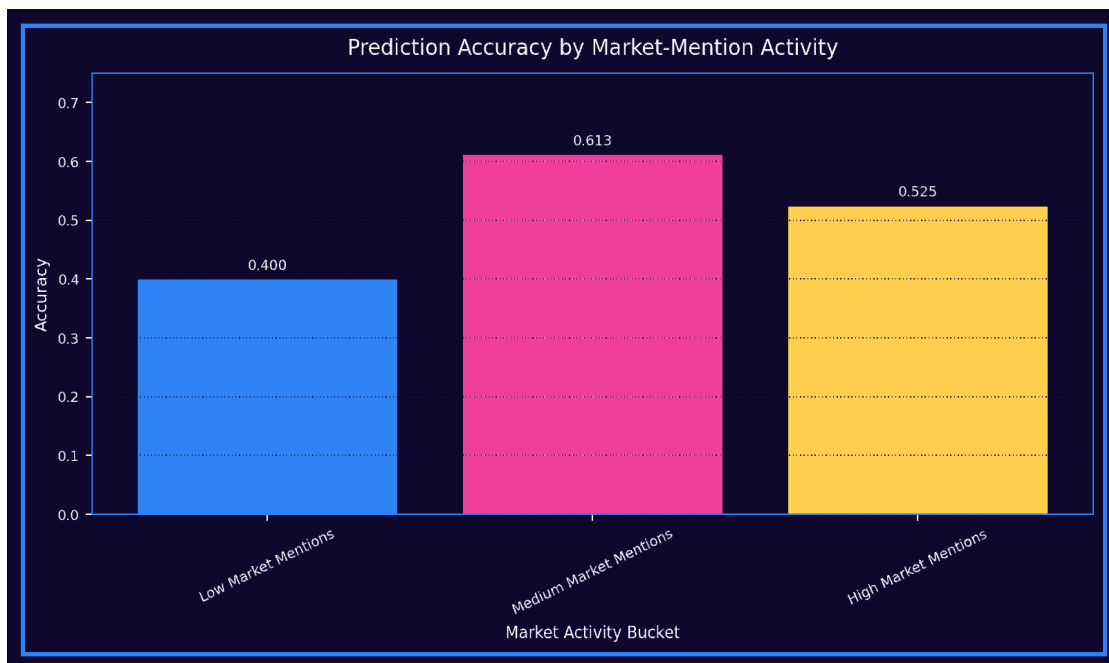


Figure 19. Prediction Accuracy by Market-Mention Activity

Code Link - Figure 20 - Power BI Model Performance Dashboard

- Exports model metrics for reporting.
- Uses classification metrics in dashboard comparison.
- Supports the model-performance dashboard page.

Notebook excerpt:

```
# Notebook Cell 43  
strong_eval_rows.append(row)  
  
strong_eval_df = pd.DataFrame(strong_eval_rows).sort_values(  
    by=['Balanced Accuracy', 'F1 Score'],
```

```

    ascending=False
)
display_dark_table(strong_eval_df)
if PLOTLY_OK:
    if probs is not None:
        try:
            row['ROC AUC'] = roc_auc_score(y_clf_test, probs)
        except Exception:
            row['ROC AUC'] = np.nan
    else:
        row['ROC AUC'] = np.nan

# Notebook Cell 46
if PLOTLY_OK:
    metric_cols = ['Accuracy', 'Balanced Accuracy', 'F1', 'ROC AUC']
    melted = clf_results_df.melt(id_vars='Model', value_vars=metric_cols, var_name='Metric', value_name='Score')
    fig = px.bar(melted, x='Model', y='Score', color='Metric', barmode='group', title='Classification ...')
    style_fig(fig, height=600).show()
else:
    clf_results_df.plot(kind='bar', x='Model', y=['Accuracy', 'Balanced Accuracy', 'F1'], title='Class...')
    plt.xticks(rotation=45)
    plt.show()

# Notebook Cell 62
OUTPUT_DIR.mkdir(exist_ok=True)

summary_cards.to_csv(OUTPUT_DIR / 'summary_cards.csv', index=False)
summary_keyword_check.to_csv(OUTPUT_DIR / 'keyword_detection_check.csv', index=False)
reg_results_df.to_csv(OUTPUT_DIR / 'regression_results.csv', index=False)
clf_results_df.to_csv(OUTPUT_DIR / 'classification_results.csv', index=False)
comparison_summary.to_csv(OUTPUT_DIR / 'baseline_comparison.csv', index=False)
corr_df.to_csv(OUTPUT_DIR / 'correlation_with_next_day_return.csv')
# ... see the notebook for the full cell

```

Description: This dashboard page summarizes model performance for business users and reinforces the weak-signal interpretation.



Figure 20. Power BI Model Performance Dashboard

The error analysis grouped the test set by market-mention activity. The best performance appeared in the medium market-mention group, not the highest activity group. This is a useful warning: more keyword activity does not automatically mean better prediction.

Overall, the results show that tweet sentiment added a limited directional signal but did not create a strong prediction system. The project is strongest when presented as an honest weak-signal experiment with a complete BI/DA workflow.

References

- DataDrivenDecision. Trump Tweets 2009-2025. Kaggle dataset. <https://www.kaggle.com/datasets/datadrivendecision/trump-tweets-2009-2025>
- Isaac Lopgu. Crude Oil Historical Data Daily Updated. Kaggle dataset. <https://www.kaggle.com/datasets/isaaclopgu/crude-oil-historical-data-daily-updated>
- HistData.com. Download Free Forex Historical Data. <https://www.histdata.com/download-free-forex-historical-data/?locale=en>
- Hutto, C. J., and Gilbert, E. VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. Proceedings of ICWSM, 2014.
- Breiman, L. Random Forests. Machine Learning, 2001.
- Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 2001.
- Scikit-learn Documentation. Machine Learning in Python. <https://scikit-learn.org/>

Power BI Documentation. Microsoft Learn. <https://learn.mic>

Appendix Expansion Overview - Extra Discussion Material

How to use these appendix pages

- Use the appendix only when the committee asks for more technical detail.
- Each appendix page connects one notebook step to the matching table, chart, dashboard, or result.
- Keep the explanation disciplined: problem, method, result, limitation, and business meaning.
- Do not present weak model scores as failure; present them as honest evidence about a noisy market signal.
- Use the code blocks to prove that the project is reproducible, not only visually documented.

Defense position

- The project is strongest as a complete BI and ML workflow, not as a trading prediction system.
- The correct claim is that sentiment was tested as an additional signal and compared against baselines.
- The main value is the pipeline: raw data to cleaned signals to models to Power BI interpretation.

Quick appendix roadmap

Appendix	What it supports in the defense
A1-A6	Project logic, setup, loading, cleaning, sentiment, and keywords
A7-A11	EDA, aggregation, oil features, merge, and safe target
A12-A18	Leakage audit, split, models, evaluation, event study, feature importance, and outputs

Appendix A1 - Defense Roadmap: How to Explain the Project

Discussion points before the code

- Start with the business question: can Trump sentiment add a measurable signal to oil movement?
- Avoid overclaiming causality; the project tests a weak signal using a controlled BI/DA workflow.
- Use this page as the speaking order for the defense: data, cleaning, NLP, features, split, models, results.

Notebook code placed before the related table / figure

```
# Defense explanation route
project_question = 'Can Trump social media sentiment improve crude-oil movement analysis?'
workflow = ['Raw data', 'Cleaning', 'Sentiment scoring', 'Keyword engineering',
           'Daily aggregation', 'Oil feature engineering', 'Date merge',
           'Chronological split', 'Regression', 'Classification',
           'EDA dashboards', 'Power BI interpretation']
interpretation_rule = 'Treat sentiment as a possible weak market signal, not as causal proof.'
print(project_question)
print(' -> '.join(workflow))
print(interpretation_rule)
```

Description: This page turns the whole notebook into a clean discussion route. It helps the presenter explain the logic before going into detailed code and results.

Extra defense notes

- Open the defense with the research question, then immediately define the boundary: analysis, not causality.
- Mention that crude oil is affected by many external drivers, so a weak signal is expected and realistic.
- Use the workflow list to show that every output in the report has a matching notebook step.
- Your strongest answer is not the accuracy number; it is the controlled comparison against baselines.
- Avoid saying tweets changed oil prices; say the project tested whether tweet sentiment added information.

Appendix A2 - Environment Setup, Libraries, and Dashboard Theme

Discussion points before the code

- Imports define the technical stack used in the project: pandas, numpy, visualization tools, NLP, and machine learning.
- The dark purple, blue, pink, and gold palette is reused to keep notebook visuals and dashboards consistent.
- This setup should run first because later charts and tables depend on these constants and imports.

Notebook code placed before the related table / figure

```
# Cell 2
# 1) Setup
# Run this cell first. If you are using Google Colab, upload the CSV files before running the notebook.

import os
import re
import sys
import math
import warnings
from pathlib import Path
from collections import Counter

warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', 160)
# ...
# Cell 4
try:
    import plotly.express as px
    import plotly.graph_objects as go
# ...
# Remaining lines continue the same notebook step.
```

Description: This code prepares the notebook environment and visual identity. It supports the charts, formatted tables, and dashboard-style outputs used throughout the report.

Why this setup matters

- The setup cell makes the notebook reproducible by loading the same libraries before any analysis starts.
- Constants such as RANDOM_STATE and TEST_SIZE make the experiment easier to repeat and defend.
- The shared dashboard color palette keeps notebook visuals, tables, and Power BI screenshots consistent.
- This cell is not just technical preparation; it defines the environment behind all later results.

Possible defense answer

- If asked why many libraries were used, explain that each one has a specific role: cleaning, NLP, modeling, or visualization.

Appendix A3 - File Paths, Loading Raw Data, and Validation

Discussion points before the code

- The project is reproducible only if the notebook clearly identifies the expected input files.
- Validation prevents silent mistakes, such as missing text columns or incorrectly named oil price fields.
- This section supports the Data Research, Acquiring Effort, and Dataset Description tables.

Notebook code placed before the related table / figure

```
# Cell 5
# 3) File Paths
# The notebook looks for the expected files in Colab, local folders, and `/mnt/data`.
# Expected files:
# - `tweets2017-2026.csv`

TWEETS_PATH_CANDIDATES = [
    '/content/tweets2017-2026.csv',
    '/content/tweets2017-2026(2).csv',
    'tweets2017-2026.csv',
    'tweets2017-2026(2).csv',
    '/mnt/data/tweets2017-2026.csv',
]

OIL_DAILY_PATH_CANDIDATES = [
    # ...
]

# Cell 6
# 4) Load Raw Data
# This cell only loads the files. Cleaning is done in later sections.

df_raw = pd.read_csv(TWEETS_PATH)
oil_daily_raw = pd.read_csv(OIL_DAILY_PATH)

oil_hourly_raw = None
# ...
# Remaining lines continue the same notebook step.
```

Description: This code loads the Trump posts, daily crude oil data, and optional hourly oil data. It also checks that the required columns exist before cleaning or modeling starts.

Why validation is necessary

- Loading files without validation is dangerous because a missing date or text column can break the whole analysis silently.
- The candidate-path logic supports both Google Colab and local execution, which makes the notebook easier to run.
- Column checks prove that the dataset structure was inspected before cleaning and modeling.
- This section supports the credibility of the data-acquisition stage.

Possible defense answer

- If asked about reproducibility, point to the file-path and validation code as the first reproducibility control.

Appendix A4 - Tweet Cleaning and Text Preparation

Discussion points before the code

- Raw tweet text includes URLs, media artifacts, symbols, and inconsistent casing.
- Text cleaning is necessary because noisy strings can distort sentiment scores and keyword counts.
- The cleaned text is the input for the Sentiment Distribution figure and keyword analysis charts.

Notebook code placed before the related table / figure

```
# Cell 9
df = df_raw.copy()

df = df.dropna(subset=['text']).copy()
df['date'] = pd.to_datetime(df['date'], errors='coerce', utc=True)
df = df.dropna(subset=['date']).copy()

optional_text_cols = ['hashtags', 'urls', 'user_mentions', 'in_reply_to', 'platform', 'handle']
for col in optional_text_cols:
    if col not in df.columns:
        df[col] = ''
    df[col] = df[col].fillna('').astype(str)

for col in ['favorite_count', 'repost_count', 'word_count', 'media_count']:
    if col not in df.columns:
        df[col] = 0
    df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)

if 'id' not in df.columns:
    df['id'] = np.arange(len(df))

# Time features
# ...
# Cell 10
# ...

# Remaining lines continue the same notebook step.
```

Description: This code creates a cleaned tweet dataset and standardizes the text field. It is directly connected to the report section on Data Primary Cleaning and Transformation.

Cleaning logic to mention

- Cleaning removes links, mentions, media artifacts, and symbols that do not help sentiment analysis.
- The goal is not to change the meaning of the posts; it is to remove noise that can distort NLP scoring.
- Date conversion is essential because the final merge with oil data depends on a clean daily date field.
- Dropping invalid text/date rows protects the model from meaningless observations.

Defense risk to avoid

- Do not say cleaning improved accuracy automatically; say it improved the quality and reliability of the inputs.

Appendix A5 - Sentiment Scoring with VADER and TextBlob Backup

Discussion points before the code

- VADER is appropriate for short social-media text because it is rule-based and handles sentiment intensity.
- TextBlob is kept as a fallback or comparison option, which makes the NLP step more defensible.
- The output sentiment columns become the core signal tested against next-day oil movement.

Notebook code placed before the related table / figure

```
# Cell 12
SENTIMENT_ENGINE = None

try:
    from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
    vader_analyzer = SentimentIntensityAnalyzer()
    SENTIMENT_ENGINE = 'VADER'
except Exception:
    try:
        import subprocess
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'vaderSentiment', '-q'])
        from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
        vader_analyzer = SentimentIntensityAnalyzer()
        SENTIMENT_ENGINE = 'VADER'
    except Exception:
        vader_analyzer = None
        SENTIMENT_ENGINE = 'TextBlob'

if SENTIMENT_ENGINE == 'TextBlob':
    try:
        from textblob import TextBlob
    except Exception:
        # ...
# Cell 13
# ...
# Remaining lines continue the same notebook step.
```

Description: This code converts cleaned text into sentiment scores and sentiment classes. These fields are later aggregated by day and merged with crude oil price data.

Sentiment explanation

- VADER converts each post into a compound sentiment score and positive/neutral/negative components.
- The sentiment label is a simplified category used for counts and dashboard interpretation.
- TextBlob is useful as a backup, but VADER is better suited for short social-media language.
- Sentiment is treated as an input feature, not as a direct explanation of oil prices.

Defense risk to avoid

- Do not claim sentiment analysis understands full political context; it is a structured approximation.

Appendix A6 - Market, Geopolitical, and Oil Keyword Engineering

Discussion points before the code

- Oil prices can respond to indirect language, not only direct mentions of oil.
- The keyword groups cover market, geopolitical, trade, sanctions, inflation, energy, and oil-related language.
- This supports the figures showing market/geopolitical/oil-related posts by year and keyword mentions.

Notebook code placed before the related table / figure

```
# Cell 15
market_keywords = [
    'oil', 'gas', 'energy', 'crude', 'crude oil', 'petroleum', 'opec',
    'saudi', 'iran', 'iraq', 'russia', 'china', 'war', 'missile',
    'sanction', 'sanctions', 'tariff', 'tariffs', 'trade', 'supply', 'supply chain',
    'shipping', 'inflation', 'prices', 'market', 'economy', 'economic',
    'jobs', 'dollar', 'fed', 'interest', 'interest rate', 'rates', 'stock', 'stocks',
    'exports', 'imports', 'manufacturing', 'pipeline', 'refinery', 'fuel'
]

geopolitical_keywords = [
    'iran', 'iraq', 'russia', 'china', 'saudi', 'opec', 'war', 'missile', 'military',
    'sanction', 'sanctions', 'tariff', 'tariffs', 'trade', 'border', 'conflict',
    'ukraine', 'nato', 'north korea', 'israel', 'gaza', 'red sea', 'houthi'
]
# ...
# Cell 16
# 9.1) Added Improvement – Expanded Keyword Coverage
# This cell extends the existing keyword system without removing the original logic. It adds more ener...

# =====
# Added Improvement: Expanded Energy + Geopolitical Keyword System
# Place: directly after the original keyword detection cell
# =====
# ...
# Remaining lines continue the same notebook step.
```

Description: This code identifies posts that are potentially market relevant. It prevents the project from being too narrow by capturing indirect economic and geopolitical signals.

Keyword engineering explanation

- The keyword groups capture direct oil terms and indirect market terms such as sanctions, tariffs, Russia, China, and inflation.
- This matters because oil markets react to geopolitical and economic language, not only to the word oil.
- Keyword counts help separate general political posts from posts that are more likely to be market relevant.
- The keyword logic supports both EDA figures and the final modeling feature set.

Possible defense answer

- If asked why indirect keywords were used, explain that oil is connected to geopolitics, trade, supply, and inflation.

Appendix A7 - Exploratory Data Analysis: Sentiment, Years, and Platforms

Discussion points before the code

- EDA checks whether the prepared tweet data behaves logically before modeling.
- The sentiment distribution shows the balance of positive, neutral, and negative posts.
- Year and platform charts explain why volume changes must be considered during interpretation.

Notebook code placed before the related table / figure

```
# Cell 18
# 11) EDA Overview
# EDA can use all available historical data. Modeling evaluation will use only the chronological test ...

summary_cards = pd.DataFrame({
    'Metric': [
        'Total Posts', 'Start Date', 'End Date', 'Sentiment Engine',
        'Average Polarity', 'Market-Related Posts', 'Direct Oil/Energy Posts'
    ],
    'Value': [
        len(df),
    ]
})
# ...
# Cell 19
if PLOTLY_OK:
    sentiment_counts = df['sentiment'].value_counts().reset_index()
    sentiment_counts.columns = ['sentiment', 'count']
    fig = px.pie(sentiment_counts, names='sentiment', values='count', title='Sentiment Distribution')
    style_fig(fig, height=450).show()
else:
    df['sentiment'].value_counts().plot(kind='pie', autopct='%1.1f%%', figsize=(6,6), title='Sentiment...')
    plt.ylabel('')
    plt.show()
# Cell 20
posts_by_year = df.groupby('year', as_index=False).agg(posts=('id', 'count'))
# ...
# Remaining lines continue the same notebook step.
```

Description: This code generates the notebook EDA outputs linked to the sentiment distribution, posts-by-year, and platform figures in the documentation.

EDA interpretation

- EDA is used before modeling to check whether the cleaned data has reasonable patterns.
- Sentiment distribution is descriptive: it shows balance between positive, neutral, and negative posts.
- Posts by year and platform help explain changes in volume and source behavior over time.
- These charts prepare the audience before the more technical model results.

Defense risk to avoid

- A sentiment pie chart alone proves nothing about oil movement; it only describes the text dataset.

Appendix A8 - Hashtags and Market-Related Tweet Visuals

Discussion points before the code

- Top hashtags provide context about repeated political and public topics in the posts.
- Market-related tweet visuals show whether direct oil language is rare compared with broader economic language.
- These figures justify using broad market keywords instead of only the word oil.

Notebook code placed before the related table / figure

```
# Cell 22
market_by_year = df.groupby('year', as_index=False).agg(
    total_posts=('id', 'count'),
    market_posts=('is_market_related', 'sum'),
    geo_posts=('is_geo_related', 'sum'),
    oil_direct_posts=('is_oil_direct_related', 'sum')
)
market_by_year['market_share'] = market_by_year['market_posts'] / market_by_year['total_posts']
display_dark_table(market_by_year)

if PLOTLY_OK:
    melted = market_by_year.melt(
        id_vars='year',
        value_vars=['market_posts', 'geo_posts', 'oil_direct_posts'],
        var_name='Type',
        value_name='Posts'
    )
    fig = px.line(melted, x='year', y='Posts', color='Type', markers=True, title='Market/Geopolitical/...')
    style_fig(fig).show()
# Cell 23
# 12) Top Hashtags
# This is descriptive only and does not enter the model unless explicitly engineered.

# ...
# Remaining lines continue the same notebook step.
```

Description: This code produces the Top Hashtags and market-related visual summaries. It supports the interpretation that the project is a broad market-sentiment study.

Hashtag and market visuals

- Hashtags give context about repeated themes in the tweet data but are not used as causal evidence.
- The market/geopolitical/oil comparison shows why the project uses a wider market-language definition.
- This section is useful for explaining why the project is broader than a simple oil-keyword search.
- The visual gap between market posts and direct oil posts supports the feature-engineering decision.

Possible defense answer

- If asked why hashtags are shown, say they are EDA context and not the main predictive variable.

Appendix A9 - Daily Tweet Aggregation for Modeling

Discussion points before the code

- The model does not learn from individual tweets directly; it learns from daily aggregated signals.
- Aggregation creates daily tweet count, sentiment averages, positive/negative tweet counts, and engagement measures.
- This table is the bridge between text analytics and the daily oil market dataset.

Notebook code placed before the related table / figure

```
# Cell 25
daily_tweets = df.groupby('date_day').agg(
    avg_sentiment=('sentiment_score', 'mean'),
    avg_polarity=('polarity', 'mean'),
    avg_subjectivity=('subjectivity', 'mean'),
    avg_positive_score=('positive_score', 'mean'),
    avg_neutral_score=('neutral_score', 'mean'),
    avg_negative_score=('negative_score', 'mean'),
    tweet_count=('text', 'count'),
    negative_tweets=('sentiment', lambda x: (x == 'Negative').sum()),
    positive_tweets=('sentiment', lambda x: (x == 'Positive').sum()),
    neutral_tweets=('sentiment', lambda x: (x == 'Neutral').sum()),
    total_likes=('favorite_count', 'sum'),
    total_reposts=('repost_count', 'sum'),
    total_engagement=('engagement', 'sum'),
    avg_word_count=('word_count', 'mean'),
    hashtag_count=('hashtag_count', 'sum'),
    mention_count=('mention_count', 'sum'),
    market_keyword_mentions=('market_keyword_count', 'sum'),
    geo_keyword_mentions=('geo_keyword_count', 'sum'),
    oil_direct_keyword_mentions=('oil_direct_keyword_count', 'sum'),
    market_related_posts=('is_market_related', 'sum'),
    geo_related_posts=('is_geo_related', 'sum'),
    oil_direct_related_posts=('is_oil_direct_related', 'sum'),
    # ...
# Remaining lines continue the same notebook step.
```

Description: This code groups tweets by day and creates the daily sentiment layer. It is essential because the oil dataset is daily and must be merged on the same time level.

Daily aggregation explanation

- The model predicts daily oil behavior, so tweet-level posts must be summarized into daily features.
- Aggregation creates daily averages, totals, and counts that can be merged with daily oil records.
- This step avoids treating thousands of individual tweets as separate oil-market observations.
- The daily table is the bridge between NLP text features and market time-series data.

Defense risk to avoid

- Do not say one tweet equals one oil-price observation; the model uses daily aggregated signals.

Appendix A10 - Daily Oil Feature Engineering

Discussion points before the code

- Oil data needs returns, volatility, lag, and rolling features before it becomes useful for machine learning.
- These features create the oil-only baseline used to test whether sentiment adds extra value.
- The code supports the Dataset Description and Feature Engineering sections of the report.

Notebook code placed before the related table / figure

```
# Cell 27
def normalize_colnames(data):
    data = data.copy()
    data.columns = [str(c).strip() for c in data.columns]
    return data

def find_col(columns, candidates):
    lower_map = {str(c).lower(): c for c in columns}
    for cand in candidates:
        if cand.lower() in lower_map:
            return lower_map[cand.lower()]
    for c in columns:
        cl = str(c).lower()
        if any(cand.lower() in cl for cand in candidates):
            return c
    return None

oil_daily = normalize_colnames(oil_daily_raw)

date_col = find_col(oil_daily.columns, ['Date', 'Datetime', 'date', 'datetime'])
open_col = find_col(oil_daily.columns, ['Open', 'open'])
high_col = find_col(oil_daily.columns, ['High', 'high'])
low_col = find_col(oil_daily.columns, ['Low', 'low'])
close_col = find_col(oil_daily.columns, ['Close', 'Adj Close', 'close'])
# ...
# Remaining lines continue the same notebook step.
```

Description: This code prepares daily crude oil variables such as returns and volatility. These fields are used both for descriptive charts and for baseline predictive models.

Oil feature engineering explanation

- Oil raw prices are transformed into returns, volatility, lags, and rolling measures for modeling.
- Lag features give the model past market information without using future values.
- The oil-only baseline is necessary because sentiment should be tested against market history, not alone.
- This is the section that makes the comparison between baseline and sentiment models fairer.

Possible defense answer

- If asked why oil-only features are included, say they create a baseline to test whether sentiment adds extra value.

Appendix A11 - Date-Based Merge and Safe Next-Day Target

Discussion points before the code

- Tweet features and oil features are merged by date, not by row order.
- The target is shifted to next-day oil movement to reduce future leakage.
- This is one of the most important methodological choices in the project defense.

Notebook code placed before the related table / figure

```
# Cell 28
# 15) Merge Daily Tweets with Safe Future Oil Target
# For day `t`, the model uses:
# - tweet features from day `t`
# - oil lag features from days before `t`

daily_merged = pd.merge(daily_tweets, oil_model_daily, on='day', how='inner')
daily_merged = daily_merged.sort_values('day').dropna(subset=['target_next_day_return_pct']).copy()

# Tweet lag features.
tweet_lag_base_cols = [
    'avg_sentiment', 'avg_polarity', 'tweet_count', 'negative_tweets', 'positive_tweets',
    'market_keyword_mentions', 'geo_keyword_mentions', 'oil_direct_keyword_mentions',
    'market_related_posts', 'geo_related_posts', 'oil_direct_related_posts',
    'total_engagement', 'total_final_impact', 'total_market_importance', 'total_oil_direct_importance'
]

tweet_lag_base_cols = [c for c in tweet_lag_base_cols if c in daily_merged.columns]

for lag in [1, 2, 3, 5, 7, 14]:
    for col in tweet_lag_base_cols:
        daily_merged[f'{col}_lag{lag}'] = daily_merged[col].shift(lag)

# ...
# ...
# Remaining lines continue the same notebook step.
```

Description: This code creates the final daily modeling dataset and defines the prediction target. It supports the Dataset Merging and target-definition explanation in the report.

Merge and target explanation

- The merge is based on the daily date field so tweet signals and oil records refer to the same day.
- The prediction target is shifted to next-day oil movement to reduce leakage and make the task more realistic.
- The model uses day t information to predict day $t+1$ behavior, not the same-day movement after knowing the close.
- This design is critical for defending the project methodology.

Defense risk to avoid

- Do not confuse the merge date with the prediction target; same-day features predict next-day outcome.

Appendix A12 - Leakage Audit and Chronological Train/Test Split

Discussion points before the code

- A chronological split is more realistic than a random split because market data has time order.
- The leakage audit checks that target-like future information is not accidentally included as a predictor.
- This section is critical for defending the honesty of the model results.

Notebook code placed before the related table / figure

```
# Cell 30
# 16) Leakage Audit
# This blocks the most dangerous mistake: using current-day/future oil columns as predictors.

leakage_forbidden_features = [
    'oil_open', 'oil_high', 'oil_low', 'oil_close', 'oil_volume',
    'oil_return_pct_same_day', 'oil_delta_same_day', 'oil_volatility_same_day',
    'target_next_day_return_pct', 'target_next_day_direction', 'target_next_day'
]

print('Forbidden raw/current/future oil columns that must NOT be in X:')
print(leakage_forbidden_features)
# Cell 31
# 17) Chronological Train/Test Split
# Time series data must be split by time. Older dates train the model; newer dates test it.

def chronological_train_test_split(data, date_col='day', test_size=0.2):
    data = data.sort_values(date_col).copy()
    split_idx = int(len(data) * (1 - test_size))
    train = data.iloc[:split_idx].copy()
    test = data.iloc[split_idx:].copy()
    if len(train) == 0 or len(test) == 0:
        raise ValueError('Train or test split is empty. Check dataset size or test_size.')
    if train[date_col].max() >= test[date_col].min():
        # ...
    # Remaining lines continue the same notebook step.
```

Description: This code validates the feature set and splits the data into training and testing periods. It supports the Train/Test Split table and the modeling methodology.

Leakage and split explanation

- Leakage happens when future information accidentally enters the training features and makes results look better than they are.
- The forbidden-feature list blocks current/future oil variables from being used incorrectly.
- Chronological splitting respects the time order of financial data and gives a more honest test.
- This page is one of the strongest methodological defense points in the project.

Possible defense answer

- If asked why not random split, say random splitting can mix future periods into training and exaggerate performance.

Appendix A13 - Feature Sets and Safe X/y Preparation

Discussion points before the code

- Separate feature lists allow fair comparison between oil-only baseline models and sentiment-based models.
- The prepared X and y objects define what the model sees and what it predicts.
- This step keeps the modeling pipeline organized and repeatable.

Notebook code placed before the related table / figure

```
# Cell 34
baseline_features = [
    c for c in daily_merged.columns
    if c.startswith('oil_return_lag')
    or c.startswith('oil_close_lag')
    or c.startswith('oil_volume_lag')
    or c.startswith('oil_volatility_lag')
    or c.startswith('oil_return_roll')
    or c.startswith('oil_volatility_roll')
]

same_day_tweet_features = [
    'avg_sentiment', 'avg_polarity', 'avg_subjectivity', 'avg_positive_score', 'avg_neutral_score', 'a...
    'tweet_count', 'negative_tweets', 'positive_tweets', 'neutral_tweets',
    'negative_ratio', 'positive_ratio', 'neutral_ratio',
    'total_likes', 'total_reposts', 'total_engagement',
    'avg_word_count', 'hashtag_count', 'mention_count',
    'market_keyword_mentions', 'geo_keyword_mentions', 'oil_direct_keyword_mentions',
    'market_related_posts', 'geo_related_posts', 'oil_direct_related_posts',
    'market_related_ratio', 'geo_related_ratio', 'oil_direct_related_ratio',
    'avg_final_impact', 'total_final_impact',
    'avg_market_importance', 'total_market_importance',
    # ...
# Cell 35
# ...
# Remaining lines continue the same notebook step.
```

Description: This code defines the predictors and target variables for regression and classification. It makes the baseline versus sentiment comparison transparent.

Feature-set explanation

- The baseline feature set contains oil-history variables only.
- The sentiment feature set adds NLP, keyword, engagement, and lagged tweet indicators.
- Using two feature sets allows the project to answer the real question: did sentiment add value beyond oil history?
- Preparing X and y clearly makes the modeling process easier to audit and repeat.

Defense risk to avoid

- Do not compare models that were trained on unclear or inconsistent feature sets.

Appendix A14 - Regression Models for Next-Day Oil Return

Discussion points before the code

- Regression predicts the size of next-day oil return, which is harder than predicting only direction.
- Multiple models are compared with a dummy benchmark to avoid exaggerating performance.
- Weak regression results are still academically useful because they show realistic limitations.

Notebook code placed before the related table / figure

```
# Cell 37
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import Ridge, HuberRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

regression_models = {
    'Dummy Mean': ('baseline', DummyRegressor(strategy='mean')),
    'Oil Baseline Ridge': ('baseline', Ridge(alpha=1.0, random_state=RANDOM_STATE)),
    'Oil Baseline RF': ('baseline', RandomForestRegressor(n_estimators=250, max_depth=5, min_samples_l...
    'Sentiment Ridge': ('sentiment', Ridge(alpha=1.0, random_state=RANDOM_STATE)),
    'Sentiment Huber': ('sentiment', HuberRegressor(max_iter=500)),
    'Sentiment RF': ('sentiment', RandomForestRegressor(n_estimators=250, max_depth=5, min_samples_lea...
    'Sentiment GB': ('sentiment', GradientBoostingRegressor(n_estimators=150, max_depth=2, learning_ra...
}

reg_results = []
reg_predictions = {}

for name, (feature_type, model) in regression_models.items():
    if feature_type == 'baseline':
        model.fit(Xb_train_scaled, y_reg_train)
# ...
# Cell 38
# ...
# Remaining lines continue the same notebook step.
```

Description: This code trains regression models and evaluates MAE, RMSE, and R2. It directly supports the Regression Results table and R2 comparison figures.

Regression interpretation

- Regression tries to predict the size of next-day oil return, which is a difficult and noisy task.
- Using Dummy Mean prevents overclaiming because it asks whether complex models beat a simple average prediction.
- Weak R2 values mean the model does not explain much of the return-size variation.
- This result is still valuable because it shows the limits of sentiment features in a realistic market setting.

Possible defense answer

- If asked why R2 is weak, say oil returns are driven by many variables not included in the tweet dataset.

Appendix A15 - Classification Models for Next-Day Direction

Discussion points before the code

- Classification predicts whether next-day oil movement is up or down/flat.
- Balanced accuracy is important because normal accuracy can hide class imbalance.
- The Sentiment Logistic model is interpreted cautiously as a small directional improvement, not a strong trading model.

Notebook code placed before the related table / figure

```
# Cell 40
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score, balanced_accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix, roc_auc_score
)

classification_models = {
    'Dummy Majority': ('baseline', DummyClassifier(strategy='most_frequent')),
    'Oil Baseline Logistic': ('baseline', LogisticRegression(max_iter=2000, class_weight='balanced', r...
    'Oil Baseline RF': ('baseline', RandomForestClassifier(n_estimators=250, max_depth=5, min_samples_...
    'Sentiment Logistic': ('sentiment', LogisticRegression(max_iter=2000, class_weight='balanced', ran...
    'Sentiment RF': ('sentiment', RandomForestClassifier(n_estimators=250, max_depth=5, min_samples_le...
    'Sentiment GB': ('sentiment', GradientBoostingClassifier(n_estimators=150, max_depth=2, learning_r...
}

clf_results = []
clf_predictions = {}
clf_probabilities = {}

# ...
# Cell 41
# ...
# Remaining lines continue the same notebook step.
```

Description: This code trains classification models and produces evaluation outputs such as accuracy, balanced accuracy, precision, recall, F1, ROC AUC, and the confusion matrix.

Classification interpretation

- Classification simplifies the task to direction: up versus down/flat next day.
- Balanced accuracy is emphasized because accuracy alone can be misleading when one class is more common.
- Sentiment Logistic showed a small improvement, but the margin is not large enough for a strong trading claim.
- The correct interpretation is limited directional signal, not reliable prediction.

Defense risk to avoid

- Do not present 51 percent accuracy as strong; present it as slightly above chance and explain the limitation.

Appendix A16 - Stronger Evaluation, Baselines, and Model Dashboard

Discussion points before the code

- The evaluation step compares all models rather than reporting only the best-looking number.
- Dummy and oil-only baselines are necessary because they show whether sentiment truly adds value.
- The model comparison dashboard converts technical metrics into a presentation-friendly view.

Notebook code placed before the related table / figure

```
# Cell 43
# =====
# Added Improvement: Stronger Quantitative Model Evaluation
# Place: directly after the best classification model report
# =====

strong_eval_rows = []

for model_name, preds in clf_predictions.items():
    probs = clf_probabilities.get(model_name)

# ...
# Cell 44
# 25) Honest Baseline Comparison
# This directly checks whether tweet/sentiment features add value beyond the oil-only baseline.

baseline_only = clf_results_df[clf_results_df['Model'].str.startswith('Oil Baseline')].sort_values('Ba...
sentiment_only = clf_results_df[clf_results_df['Model'].str.startswith('Sentiment')].sort_values('Bala...
dummy_only = clf_results_df[clf_results_df['Model'].str.startswith('Dummy')].head(1)

comparison_summary = pd.concat([dummy_only, baseline_only.head(1), sentiment_only.head(1)], ignore_ind...
display_dark_table(comparison_summary)

# ...
# ...
# Remaining lines continue the same notebook step.
```

Description: This code builds stronger quantitative evaluation, baseline comparison, model dashboard visuals, and correlation analysis. It supports the Model Evaluation and Dashboard Insights sections.

Evaluation and dashboard explanation

- The stronger evaluation table compares models side by side instead of choosing only the best-looking result.
- Dummy and oil-only baselines protect the project from exaggerating the contribution of sentiment.
- The dashboard version of model metrics helps non-technical viewers understand the comparison quickly.
- This page connects technical model evaluation with BI reporting.

Possible defense answer

- If asked what the dashboard adds, say it makes model comparison and limitations easier for business users to interpret.

Appendix A17 - Event Study and Clear Sentiment/Oil Visualizations

Discussion points before the code

- The event study compares oil behavior around positive and negative sentiment shock days.
- The -3 to +3 day window is descriptive and should not be presented as causal proof.
- Clear visualization scaling makes sentiment and oil-return charts easier to discuss.

Notebook code placed before the related table / figure

```
# Cell 49
# =====
# Added Improvement: Event Study Around Strong Sentiment Shock Days
# Place: directly after the correlation analysis cell
# =====

event_df = daily_merged.copy().sort_values('day').reset_index(drop=True)

# Define strong sentiment events using the 10th and 90th percentiles.
negative_event_threshold = event_df['avg_sentiment'].quantile(0.10)
positive_event_threshold = event_df['avg_sentiment'].quantile(0.90)
# ...
# Cell 50
# 28) Fixed Visualization Scaling
# Sentiment and oil return are not on the same scale. This chart uses z-scores for visual comparison o...

def zscore(s):
    s = pd.Series(s)
    std = s.std()
    if std == 0 or pd.isna(std):
        return s * 0
    return (s - s.mean()) / std

# ...
# ...
# Remaining lines continue the same notebook step.
```

Description: This code creates event-study figures and normalized visual comparisons. It supports the Event Study Results section and the caution that the signal is weak.

Event-study interpretation

- The event study investigates behavior around strong sentiment days using a -3 to +3 day window.
- It is exploratory and descriptive; it should not be used as proof that tweets caused oil movement.
- The visual is useful because it shows whether patterns around sentiment shocks are consistent or unstable.
- The strongest statement is that negative shocks showed noticeable movement, but the evidence remains limited.

Defense risk to avoid

- Never say the event study proves causality; say it helps inspect market behavior around selected sentiment windows.

Appendix A18 - Feature Importance, Error Analysis, Hourly Check, and Saving Outputs

Discussion points before the code

- Feature importance explains which variables the tree-based model used most heavily.
- Prediction timeline and error buckets show where the model succeeds and where it fails.
- Saving outputs makes the project usable for Power BI reporting and final documentation.

Notebook code placed before the related table / figure

```
# Cell 55
# 29) Feature Importance for Tree-Based Sentiment Model
# Feature importance is not causality. It only shows what the model used for prediction.

# Choose the best tree-based sentiment classifier if available.
tree_model_name = None
for candidate in ['Sentiment RF', 'Sentiment GB']:
    if candidate in classification_models:
# ...
# Cell 56
# 30) Prediction Timeline for Best Classifier
# This helps visually inspect when the model gets direction right or wrong.

test_predictions_df = test_df[['day', 'target_next_day', 'target_next_day_return_pct', 'target_next_da...
test_predictions_df['predicted_direction'] = best_clf_pred
test_predictions_df['correct'] = (test_predictions_df['predicted_direction'] == test_predictions_df['t...

# ...
# Cell 57
# 31) Error Analysis by Market Activity
# Does the classifier perform better on days with more market-related tweets?

error_analysis_df = test_predictions_df.merge(
    daily_merged[['day', 'tweet_count', 'market_keyword_mentions', 'geo_keyword_mentions', 'oil_direct...
# ...
# Remaining lines continue the same notebook step.
```

Description: This final appendix code supports feature-importance figures, correct-versus-incorrect prediction plots, market-activity error analysis, optional hourly context, final summary, and saved report outputs.

Final technical takeaway

- Feature importance identifies which variables the tree-based model used most, but it is not causal importance.
- Error analysis shows when the model was right or wrong, which is more honest than reporting accuracy only.
- Hourly analysis is an additional check, not the main final conclusion of the project.
- Saving outputs to CSV makes the project ready for Power BI refresh and final documentation.

Defense closing line

- The project is successful because it builds a complete, reproducible BI and ML pipeline and reports weak results honestly.

References

Reference page added at the end of the discussion PDF

These sources support the datasets, NLP sentiment method, machine learning models, and dashboard tools used in the project. They should be mentioned only as supporting references, while the actual results come from the notebook workflow and prepared reporting outputs.

Main data sources

- DataDrivenDecision. Trump Tweets 2009-2025. Kaggle dataset.
<https://www.kaggle.com/datasets/datadrivendecision/trump-tweets-2009-2025>
- Isaac Lopgu. Crude Oil Historical Data Daily Updated. Kaggle dataset.
<https://www.kaggle.com/datasets/isaaclopgu/crude-oil-historical-data-daily-updated>
- HistData.com. Download Free Forex Historical Data. <https://www.histdata.com/download-free-forex-historical-data/?locale=en>

Methods and modeling references

- Hutto, C. J., and Gilbert, E. VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. Proceedings of ICWSM, 2014.
- Breiman, L. Random Forests. Machine Learning, 2001.
- Friedman, J. H. Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 2001.
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011.

Tools and documentation references

- Scikit-learn Documentation. Machine Learning in Python. <https://scikit-learn.org/>
- Pandas Documentation. Data analysis and manipulation tools. <https://pandas.pydata.org/docs/>
- Plotly Documentation. Interactive visualizations in Python. <https://plotly.com/python/>
- Microsoft Learn. Power BI Documentation. <https://learn.microsoft.com/power-bi/>

Defense note

Use the references to support the source credibility and methods. Do not present them as proof that tweets caused oil movements. The project conclusion should stay realistic: sentiment provided a limited directional signal, while oil prices remain affected by many external market factors.